



Politechnika Krakowska im. Tadeusza Kościuszki

Wydział Informatyki i Telekomunikacji

PRACA MAGISTERSKA

Zastosowanie fine-tuning'u dużych modeli językowych do wykrywania niechcianej poczty elektronicznej

Title consistent with the topic/field of the thesis

Autor:	Wojciech Drózdź
Kierunek:	Informatyka
Opiekun pracy:	dr inż. Dariusz Żelasko

Kraków, 2026

Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań. Tutaj możesz umieścić treść podziękowań.

Streszczenie

Streszczenie po polsku ...

Abstract

Abstract in English . . .

Spis treści

Spis rysunków	xi
Spis tabel	xiii
Lista kodów źródłowych	xvii
1. Część badawcza	1
1.1. Wybór modelu językowego z otwartymi wagami	1
1.2. Metodologia badawcza	3
1.2.1. Analiza wykorzystanych zbiorów danych	3
1.2.2. Przygotowanie danych	9
1.2.3. Testowane sposoby dostrajania modelu	15
1.2.4. Ocena modelu bazowego bez dostrajania	15
1.2.5. Dobór parametrów treningu	17
1.2.6. Porównanie metod dostrajania	28
Bibliografia	35

Zawartość spisu treści — tytuły rozdziałów oraz ich liczba zależą od tematyki pracy — należy ustalić z opiekunem pracy.

Spis rysunków

1.1. Porównanie liczności dostępnych surowych zbiorów danych. Skala pozioma jest logarytmiczna.	5
1.2. Rozkład klas w surowych korpusach. Klasa pozytywna oznacza spam, phishing lub wiadomość oszukańczą.	6
1.3. Udział źródeł w najczęściej używanym zbiorze treningowym.	8
1.4. Liczba rekordów przed deduplikacją oraz po deduplikacji.	11
1.5. Rozmiary grup duplikatów po połączeniu zbiorów danych poza enron	12
1.6. Rozkład długości treści wiadomości według etykiety w analizowanym wariancie danych.	13
1.7. Terminy najsilniej powiązane z klasą pozytywną według wygładzonego ilorazu szans.	14
1.8. Przebieg funkcji straty dla wybranych konfiguracji treningu. Wykres <i>training loss</i> został wygładzony medianą kroczącą, aby ograniczyć szum wynikający z pomiaru na kolejnych batchach.	19
1.9. Przebieg F1 oraz <i>accuracy</i> na zbiorze walidacyjnym dla wybranych konfiguracji treningu.	20
1.10. Czas treningu poszczególnych konfiguracji. Kolor słupka oznacza maksymalną długość kontekstu. Porównanie obejmuje 15 treningów, dla których zapisano metrykę train_runtime	20
1.11. Najlepszy wynik zewnętrzny uzyskany przez każdą analizowaną konfigurację metody klasyfikacji następnego tokenu.	22
1.12. Porównanie metryk dla pięciu najlepszych konfiguracji według wyniku zewnętrznego.	23
1.13. Przebieg <i>accuracy</i> dla analizowanych konfiguracji. Każdy panel odpowiada jednej konfiguracji, a linie przedstawiają osobne zbiory ewaluacyjne.	24
1.14. Luka generalizacji między F1 na train_subset a wynikiem zewnętrznym dla najlepszych punktów kontrolnych poszczególnych konfiguracji.	25
1.15. Przebieg wyniku zewnętrznego dla pięciu najlepszych konfiguracji w kolejnych ocenianych punktach kontrolnych.	26
1.16. Odsetki błędów typu <i>false positive</i> i <i>false negative</i> dla wybranego punktu kontrolnego konfiguracji K08.	27

1.17. Porównanie najlepszych wariantów metod na czterech zbiorach ewaluacyjnych. Dla <code>train_subset</code> i <code>spam_ham</code> pokazano F1, dla <code>enron</code> <i>specificity</i> , a dla <code>fraudulent_email_corpus</code> <i>recall</i>	29
1.18. Porównanie błędów typu <i>false positive</i> na zbiorze <code>enron</code> oraz <i>false negative</i> na zbiorze <code>fraudulent_email_corpus</code> dla najlepszych wariantów metod. Punkty położone bliżej lewego dolnego rogu mają korzystniejszy profil błędów.	30
1.19. Przebieg F1 na <code>train_subset</code> oraz wyniku zewnętrznego S_{ext} dla najlepszej konfiguracji każdej metody. Linia przerywana oznacza punkt kontrolny o najwyższym wyniku zewnętrznym.	32

Spis tabel

1.1. Porównanie małych rodzin modeli językowych rozważanych jako baza dla klasyfikatora.	2
1.2. Porównanie wybranych modeli w benchmarkach dostępnych dla wszystkich rozważanych rodzin.	3
1.3. Charakterystyka zbiorów danych	4
1.4. Dostępne kolumny w surowych zbiorach danych oraz sposób ich parsowania .	9
1.5. Wyniki modelu bazowego bez dostrajania. Wartości podano w procentach. .	16
1.6. Konfiguracje treningu objęte analizą punktów kontrolnych.	18
1.7. Ranking najlepszych punktów kontrolnych dla konfiguracji metody klasyfikacji następnego tokenu. Wynik zewnętrzny jest średnią z <i>specificity</i> na enron , <i>recall</i> na fraudulent_email_corpus oraz F1 na spam_ham	21
1.8. Najlepsze punkty kontrolne uzyskane dla porównywanych metod. Gwiazdka przy metodzie 04 oznacza wynik częściowy, obejmujący dwie ocenione konfiguracje.	28
1.9. Moment uzyskania najlepszego wyniku zewnętrznego dla najlepszej konfiguracji każdej metody.	31

Lista algorytmów

Lista kodów źródłowych

1.1. Prompt wykorzystany do oceny modelu bazowego w wariancie generowania i parsowania.	16
---	----

1. Część badawcza

1.1. Wybór modelu językowego z otwartymi wagami

Model bazowy dobierano z myślą o eksperymentach, które trzeba było wielokrotnie powtarzać. Zbyt duży model podnosiłby koszt każdego treningu, wydłużał ewaluację i zwiększał rozmiar zapisywanych punktów kontrolnych. Z tego powodu przegląd ograniczono do małych rodzin modeli dostępnych jako otwarte wagi, zgodnych z narzędziami używanymi później w pracy, takimi jak `transformers`, `peft`, `vLLM` albo `llama.cpp`. Przyjęto też praktyczne założenie, że klasyfikator powinien działać na serwerze z jedną kartą GPU, a po kwantyzacji również w słabszym środowisku.

Do porównania wybrano pięć rodzin: `Qwen3`, `Gemma 3`, `Llama 3.2`, `SmolLM2` oraz `Phi-3`. Benchmarki nie były traktowane jako bezpośredni pomiar skuteczności wykrywania spamu. Standardowe testy mierzą raczej wiedzę ogólną, rozumowanie, matematykę, kodowanie lub wykonywanie instrukcji. W tym miejscu służyły więc tylko jako orientacyjny sygnał jakości modelu przed dostrojeniem.

Rodzina	Rozważany wariant	Licencja / dostęp	Cechy techniczne	Znaczenie dla tej pracy
Qwen3	0,6B	Apache 2.0	32k tokenów kontekstu, wsparcie ponad 100 języków, tryb odpowiedzi szybkiej i tryb rozumowania[1, 2].	Mały wariant o przyjętej w pracy pojemności, z wygodnym wsparciem standardowych narzędzi.
Gemma 3	270M / 1B	otwarte wagi, licencja Gemma	Bardzo małe warianty, kontekst 32k tokenów dla modeli 270M i 1B, integracja z ekosystemem Google[3].	Niski koszt inferencji, ale mniej neutralne warunki licencyjne niż w przypadku Apache 2.0.
Llama 3.2	1B / 3B	Llama 3.2 Community License	Warianty tekstowe 1B i 3B, kontekst 128k tokenów, szerokie wsparcie narzędziowe rodziny Llama[4].	Dobrze wspierana rodzina modeli, lecz najmniejszy wariant jest większy od Qwen3-0.6B, a licencja jest niestandardowa.
SmolLM2	135M / 360M / 1,7B	Apache 2.0	Rodzina projektowana z myślą o małych modelach i zastosowaniach lokalnych; wariant 1,7B trenowany na 11 bilionach tokenów[5].	Wariant 1,7B daje wygodną bazę lokalną, ale jest prawie trzykrotnie większy od Qwen3-0.6B.
Phi-3	3,8B	otwarte wagi, licencja MIT	Model Phi-3-mini ma 3,8B parametrów i był projektowany jako mały model o wysokiej jakości ogólnej[6, 7].	Wysoka jakość ogólna okupiona większym kosztem treningu, inferencji i przechowywania punktów kontrolnych.

Tabela 1.1.: Porównanie małych rodzin modeli językowych rozważanych jako baza dla klasyfikatora.

Rodzina	Porównywany wariant	GSM8K	MMLU-Pro
Qwen3	0,6B Base	59,59[1]	24,74[1]
Gemma 3	1B IT	62,8[3]	14,7[3]
Llama 3.2	1B Instruct	44,4[4]	8,2[8]
SmolLM2	1,7B Instruct	48,2[5]	19,3[5]
Phi-3	mini 4k instruct	85,7[7]	45,66[7]

Tabela 1.2.: Porównanie wybranych modeli w benchmarkach dostępnych dla wszystkich rozważanych rodzin.

Tabela 1.2 nie daje prostego rozstrzygnięcia. Phi-3-mini ma najlepsze wyniki w obu benchmarkach, ale jest wyraźnie większy, więc podnosi koszt treningu, inferencji i przechowywania kolejnych punktów kontrolnych. Najmniejsze warianty Gemma 3 270M oraz SmolLM2 135M/360M są bardzo tanie w uruchomieniu, lecz w klasyfikacji wiadomości mogą mieć zbyt małą pojemność, szczególnie dla przykładów z pogranicza spamu, phishingu i poczty legalnej. Jako punkt startowy wybrano więc Qwen3-0.6B. Nie był to model z najwyższymi wynikami ogólnymi, ale miał rozmiar sprzyjający wielokrotnemu powtarzaniu treningów, korzystał z licencji Apache 2.0, obsługiwał format czatu, oferował kontekst 32k tokenów i działał w narzędziach używanych w dalszych eksperymentach. Wybór wynikał głównie z ograniczeń praktycznych: kosztu, powtarzalności badań i możliwości późniejszego uruchomienia klasyfikatora poza dużą infrastrukturą.

1.2. Metodologia badawcza

1.2.1. Analiza wykorzystanych zbiorów danych

Wybór i przygotowanie danych może mieć równie duży wpływ na końcowy wynik jak architektura modelu lub parametry treningu.

Jednym z głównych etapów pracy było utworzenie zbioru wiadomości elektronicznych przeznaczonego do zadania binarnej klasyfikacji. Wiadomości oznaczano jako pożądane (*ham*, klasa negatywna) albo niepożądane (*spam*, klasa pozytywna). Do klasy pozytywnej włączono spam, wiadomości oszukańcze, phishing oraz podejrzanе wiadomości marketingowe. Przyjęto taki podział, ponieważ trudno znaleźć duże korpusy, które konsekwentnie rozróżniają te podklasy.

Korpusy publiczne różnią się nie tylko liczbą rekordów, lecz także sposobem zapisu wiadomości, semantyką etykiet, obecnością metadanych oraz formatem treści. Z tego powodu analiza obejmuje ocenę pochodzenia danych, rozkładu klas, długości wiadomości, obecności duplikatów oraz potencjalnego ryzyka przecieku informacji między częścią treningową i

testową.

W pracy przyjęto, że wszystkie zbiory są sprowadzane do wspólnego schematu rekordu: `subject`, `body`, `label` oraz `source`. Pole `subject` przechowuje temat wiadomości, `body` jej treść, `label` wartość liczbowa klasy, a `source` nazwę korpusu źródłowego. Dzięki temu korpusy o różnej strukturze można porównywać bez opierania się na metadanych, które nie występują we wszystkich źródłach.

Do eksperymentów wykorzystano korpusy wymienione w tabeli 1.3.

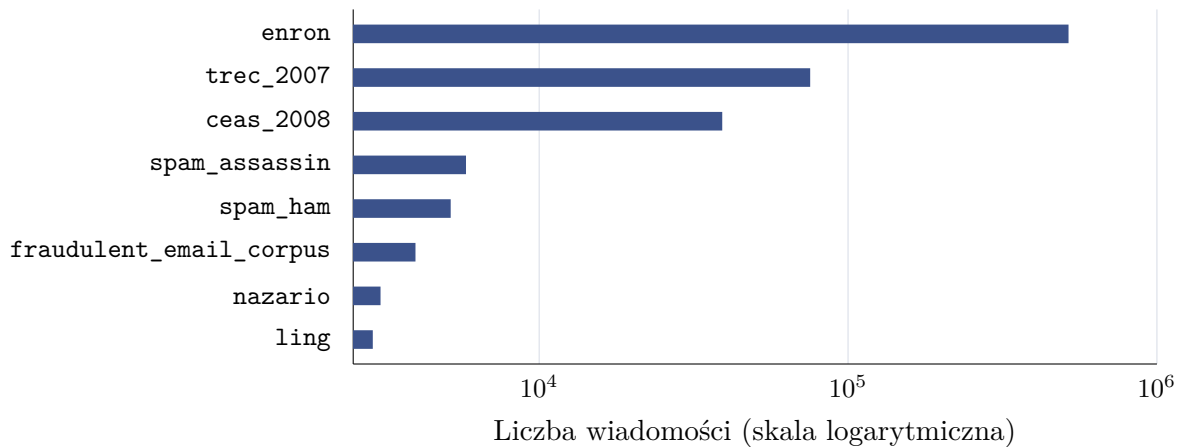
Tabela 1.3.: Charakterystyka zbiorów danych

Zbiór	Wiersze	Ham	Spam	Spam [%]	Uwagi
enron[9]	517 401	517 401	0	0,0	zbiór wiadomości pożądanych, trak- towany jako klasa negatywna
trec_2007[10]	75 419	25 220	50 199	66,6	korpus TREC 2007
ceas_2008[11]	39 154	17 312	21 842	55,8	dane CEAS 2008
spam_assassin[12]	5 796	3 900	1 896	32,7	
spam_ham[13]	5 171	3 672	1 499	29,0	
fraudulent_email[14] _corpus	3 978	0	3 978	100,0	korpus oszukańczych wiadomości phishin- gowych zwanych potocznie <i>nigeryj- skimi</i> [15]
nazario[16]	3 065	1 500	1 565	51,1	zbiór wiadomości phi- shingowych
ling[17]	2 893	2 412	481	16,6	

Łącznie analizowane zbiory surowe obejmują 652 877 wiadomości. Nie wszystkie źródła zostały jednak bezpośrednio połączone w jeden korpus treningowy. Największy zbiór, `enron`, zawiera wyłącznie wiadomości klasy negatywnej. Z kolei `fraudulent_email_corpus` zawiera wyłącznie przykłady klasy pozytywnej. Bezpośrednie wykorzystanie wszystkich danych mogłoby oznaczać, że model zamiast uczyć się różnic między treścią spamu i poczty legalnej nauczyłby się rozpoznawać pochodzenie przykładu. Przykładowo, jeśli większość wiadomości legalnych pochodziłaby z jednego archiwum firmowego, a większość wiadomości niepożądanych z innego publicznego korpusu, klasyfikator mógłby wykorzystywać artefakty konkretnego zbioru danych, a nie ogólne cechy spamu. Taki problem jest szczególnie niebezpieczny w eksperymentach z dużymi modelami językowymi, ponieważ modele te potrafią wychwytywać subtelne i nieintencjonalne wzorce formatowania[18].

Rysunek 1.1 pokazuje skalę różnic między korpusami. Połączenie wszystkich danych bez próbkowania prowadziłoby do dominacji zbioru `enron`. Taka dominacja byłaby niepożądana

nie tylko ze względu na nierównowagę klas, lecz także ze względu na pochodzenie większości danych *ham* z jednego korpusu.



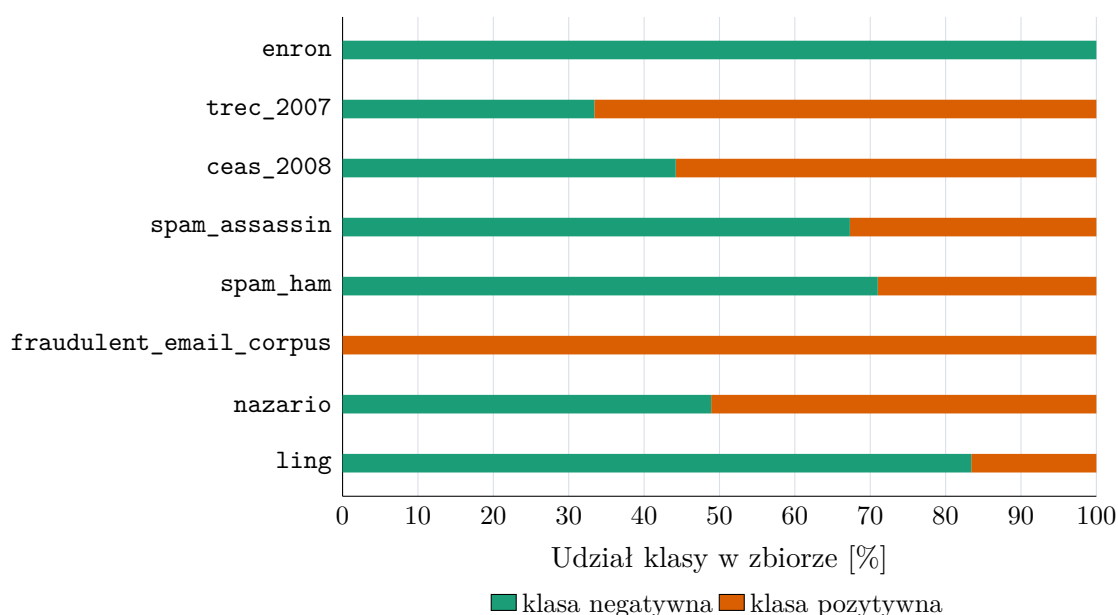
Rysunek 1.1.: Porównanie liczności dostępnych surowych zbiorów danych. Skala pozioma jest logarytmiczna.

Rysunek 1.2 przedstawia rozkład klas w źródłowych korpusach. Widoczne są trzy typy zbiorów. Pierwszą grupę tworzą zbiory względnie zbalansowane, takie jak *nazario* oraz *ceas_2008*. Drugą grupę tworzą zbiory o wyraźnej przewadze jednej z klas, na przykład *trec_2007*, *ling*, *spam_assassin* i *spam_ham*. Trzecią grupę stanowią korpusy jednoklasowe: *enron* oraz *fraudulent_email_corpus*. Dla modelowania klasyfikacyjnego najbezpieczniejsze są zbiory, które zawierają obie klasy i w których rozkład klas nie jest skrajnie zaburzony. Zbiory jednoklasowe mogą być użyteczne jako materiał pomocniczy, ale wymagają ostrożnego łączenia z innymi źródłami, aby nie stworzyć sztucznego zadania rozpoznawania korpusu zamiast rozpoznawania spamu.

Charakterystyka poszczególnych źródeł

ENRON

Zbiór *enron* jest największym z wybranych źródeł danych. Zawiera 517 401 wiadomości, z których wszystkie przynależą do klasy negatywnej. Jego zaletą jest duża liczność i naturalny charakter korespondencji, jednak jednocześnie jest to zbiór specyficzny domenowo. Pochodzi z archiwum organizacyjnego, dlatego może zawierać wiele powtarzalnych struktur, tematów, podpisów, cytowań oraz wątków wewnętrznych. Włączenie go w całości do zbioru treningowego mogłoby sztucznie zwiększyć liczbę przykładów prawdziwej korespondencji, ale niekoniecznie poprawiłoby zdolność modelu do ogólnego rozpoznawania wiadomości pożądanych. Z tego powodu w analizie końcowej nie wybrano wariantu opartego na pełnym zbiorze *enron*.



Rysunek 1.2.: Rozkład klas w surowych korpusach. Klasa pozytywna oznacza spam, phishing lub wiadomość oszukańczą.

LING

Zbiór **ling** obejmuje 2 893 rekordy: 2 412 wiadomości pożądaných oraz 481 przykładów klasy pozytywnej. Jest znacznie mniejszy od głównych korpusów, ale ma przejrzysty format z osobnym tematem, treścią i etykietą.

NAZARIO

Zbiór **nazario** obejmuje 3 065 rekordów i jest prawie zbalansowany: 1 500 wiadomości pożądaných oraz 1 565 przykładów klasy pozytywnej. Jego zaletą jest obecność pól tematu, treści oraz dodatkowych metadanych, takich jak nadawca, odbiorca, data i adresy URL.

TREC_2007

Zbiór **trec_2007** zawiera 75 419 wiadomości, z czego 50 199 należy do klasy pozytywnej, a 25 220 do klasy negatywnej. Jego zaletą jest relatywnie duża liczność i obecność surowej treści wiadomości. Rozkład klas jest jednak przesunięty w kierunku spamu, który stanowi około 66,6% rekordów.

CEAS_2008

Zbiór **ceas_2008** zawiera 39 154 wiadomości i również ma przewagę klasy pozytywnej, ale jest ona mniejsza niż w **trec_2007**. W zbiorze znajduje się 21 842 wiadomości spamowych oraz 17 312 prawdziwych. Istotną zaletą tego korpusu jest obecność pól takich jak temat, treść, nadawca, odbiorca, data i adresy URL. Dla niniejszej pracy najważniejsze są jednak temat i treść, ponieważ celem jest sprawdzenie możliwości dostrajania modelu językowego do klasyfikacji na podstawie tekstu wiadomości. Obecność adresów URL jest cenna analitycznie,

ale nie wszystkie korpusy posiadają to pole, dlatego w podstawowym schemacie zostało ono pominięte.

FRAUDULENT_EMAIL_CORPUS

Zbiór `fraudulent_email_corpus` dotyczy wiadomości oszukańczych. Jest jednoklasowy i zawiera 3 978 wiadomości traktowanych jako klasa pozytywna. Dane te są wartościowe z punktu widzenia semantycznej różnorodności spamu, ponieważ wiadomości oszukańcze mogą różnić się od typowych reklam farmaceutycznych, newsletterów czy masowego phishingu.

SPAM_ASSASSIN I SPAM_HAM

Zbiory `spam_assassin` i `spam_ham` są mniejsze, ale dobrze pasują do klasycznego zadania klasyfikacji spam/ham. `spam_assassin` zawiera 5 796 rekordów, a `spam_ham` 5 171. W obu przypadkach klasa pozytywna jest mniejszością. Reprezentują inny format i historię przygotowania danych niż korpusy CEAS i TREC użyte w treningu.

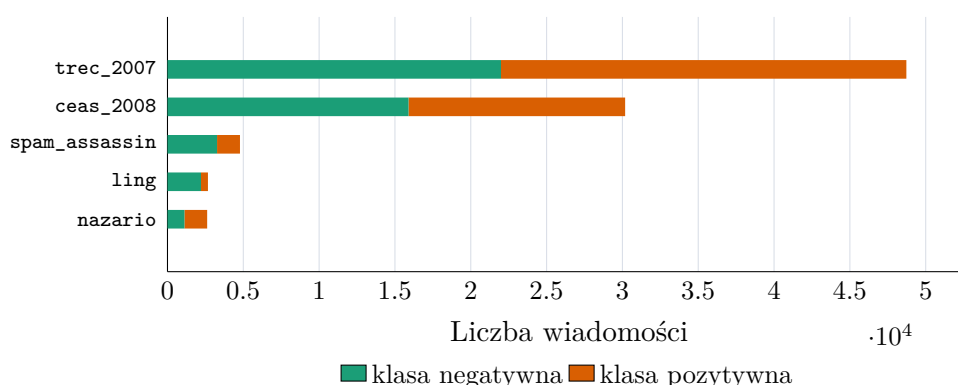
Wybór głównego zbioru do eksperymentów

W eksperymentach nie korzystano z jednego niezmiennego wariantu danych. Najczęściej używanym zbiorem treningowym był korpus zbudowany z połączenia wszystkich źródeł poza `enron`, `fraudulent_email_corpus` oraz `spam_ham`. W części eksperymentów stosowano także węższe warianty oparte na zbiorach `trec_2007`, `ceas_2008` albo ich połączeniu. Dzięki temu można było sprawdzić model zarówno na większym, bardziej zróżnicowanym korpusie, jak i na danych pochodzących z bardziej jednorodnych źródeł.

Niektóre zbiory celowo wykluczono z treningu. `enron` pomijano ze względu na bardzo dużą licznosc i jednoklasowy charakter. Jego włączenie mogłoby zdominować klasę negatywną oraz sprawić, że model uczyłby się specyfiki wewnętrznej korespondencji jednej organizacji zamiast ogólnych cech wiadomości legalnych. Z kolei `fraudulent_email_corpus` pozostawiono poza treningiem ze względu na jednoklasowość oraz potrzebę posiadania całkowicie niezależnego zbioru do sprawdzenia generalizacji na wiadomościach oszukańczych. `spam_ham` był traktowany podobnie: jako prosty, zewnętrzny punkt odniesienia dla klasycznego zadania spam/ham.

Po zbalansowaniu najczęściej używany wariant treningowy zawierał 88 970 wiadomości: 44 485 przykładów klasy negatywnej i 44 485 przykładów klasy pozytywnej. Zbiór powstał po usunięciu pustych rekordów, zastosowaniu agresywnej deduplikacji treści oraz próbkowaniu do proporcji 50/50. W składzie źródłowym dominowały `trec_2007` i `ceas_2008`, uzupełnione o `spam_assassin`, `ling` oraz `nazario`.

Rysunek 1.3 przedstawia skład źródłowy najczęściej używanego zbioru treningowego. Balans klas nie oznacza tu balansu źródeł: najwięcej rekordów pochodzi z `trec_2007`. Pole `source` zachowano w danych analitycznych, aby po treningu sprawdzać wyniki oddzielnie dla poszczególnych korpusów. Duża różnica skuteczności między źródłami oznaczałaby ograniczoną odporność na zmianę dystrybucji danych.



Rysunek 1.3.: Udział źródeł w najczęściej używanym zbiorze treningowym.

Zbiór treningowy

Zbiór treningowy służył do aktualizacji parametrów modelu w procesie fine-tuningu i stanowił 80% całego przygotowanego wariantu danych.

Podobnie jak pozostałe podzbiory, był wydzielany dopiero po normalizacji, oczyszczeniu, deduplikacji i balansowaniu wybranego wariantu danych. Dzięki temu każda konfiguracja eksperymentalna korzystała ze spójnego podziału, a ta sama lub bardzo podobna wiadomość nie mogła trafić jednocześnie do zbioru treningowego oraz testowego. Zbiór treningowy zawierał przykłady obu klas w równych proporcjach, ponieważ wcześniej zastosowano balansowanie do relacji 50/50.

Zbiór walidacyjny

Zbiór walidacyjny stanowiący 10% całego zbioru był używany do podejmowania decyzji w trakcie eksperymentów: wyboru liczby epok, konfiguracji hiperparametrów, momentu zatrzymania treningu oraz porównania wariantów promptu lub formatu odpowiedzi. Ponieważ w pracy analizowane są modele językowe, zbiór walidacyjny pełni także rolę praktycznej kontroli formatu generowanej odpowiedzi. Gdy model miał zwracać etykietę w określonej strukturze, na przykład jako tekst `spam/ham` albo obiekt JSON, na zbiorze walidacyjnym sprawdzano, czy odpowiedzi były poprawnie parsowalne.

Zbiór testowy

Zbiór testowy pozostał odseparowany do samego końca eksperymentów i stanowił 10% przygotowanego wariantu danych. Nie był używany do strojenia hiperparametrów ani do podejmowania decyzji w trakcie treningu. Jego rolą była końcowa ocena skuteczności na przykładach pochodzących z tej samej dystrybucji co zbiór treningowy. Oprócz tego model

sprawdzano także na zbiorach nieużywanych w treningu, aby ocenić odporność na zmianę źródła danych.

1.2.2. Przygotowanie danych

Normalizacja schematu

Pierwszym krokiem przygotowania danych było sprowadzenie różnych formatów źródłowych do jednego schematu. Niektóre pliki zawierają osobne kolumny **subject** i **body**; inne przechowują całą wiadomość w jednym polu tekstowym; jeszcze inne mają format zbliżony do surowego e-maila z nagłówkami. Przyjęto zasadę zachowania możliwie dużej liczby przykładów. W przypadku wiadomości surowych wydzielano temat oraz treść. Jeżeli wiadomość była wieloczęściowa, analizowano część tekstową. Jeżeli nie udało się poprawnie sparsować struktury, całą dostępną treść traktowano jako ciało wiadomości.

Drugim elementem normalizacji było ujednolicenie etykiet. W różnych zbiorach klasa pozytywna bywa oznaczana jako **spam**, **1**, **Class=1** albo przez sam fakt pochodzenia z korpusu oszukańczego. W pracy przyjęto konwencję, że **label=1** oznacza wiadomość niepożądaną, phishingową lub oszukańczą, natomiast **label=0** oznacza wiadomość prawdziwą.

Tabela 1.4.: Dostępne kolumny w surowych zbiorach danych oraz sposób ich parsowania

Zbiór	Dostępne kolumny	Sposób parsowania
enron	file, message	Wydzielenie tematu i treści z pola message .
trec_2007	label, origin	Wydzielenie tematu i treści z pola origin .
ceas_2008	sender, receiver, date, subject, body, label, urls	Bezpośrednie mapowanie pól subject , body , label .
spam_assassin	text, target	Wydzielenie tematu i treści z pola text .
spam_ham	label, text, label_num	Wydzielenie tematu z prefiksu Subject: .
fraudulent_email_corpus	wiadomości zapisane kolejno z nagłówkami	Wydzielenie tematu z nagłówków i treści z wiadomości.
nazario	sender, receiver, date, subject, body, label, urls	Bezpośrednie mapowanie pól subject , body , label .

Zbiór	Dostępne kolumny	Sposób parsowania
ling	subject, message, label	Przeniesienie message do pola body

Usuwanie pustych rekordów

Po normalizacji odrzucano rekordy, w których puste były jednocześnie temat i treść. Przykład pozbawiony jakiegokolwiek tekstu wnosi niewiele do uczenia, a może wprowadzać artefakty techniczne. Jeżeli pewien korpus zawierałby dużo pustych rekordów tylko w jednej klasie, model mógłby nauczyć się, że brak danych jest cechą klasy, co nie byłoby wiarygodną regułą w realnym wdrożeniu.

Deduplikacja

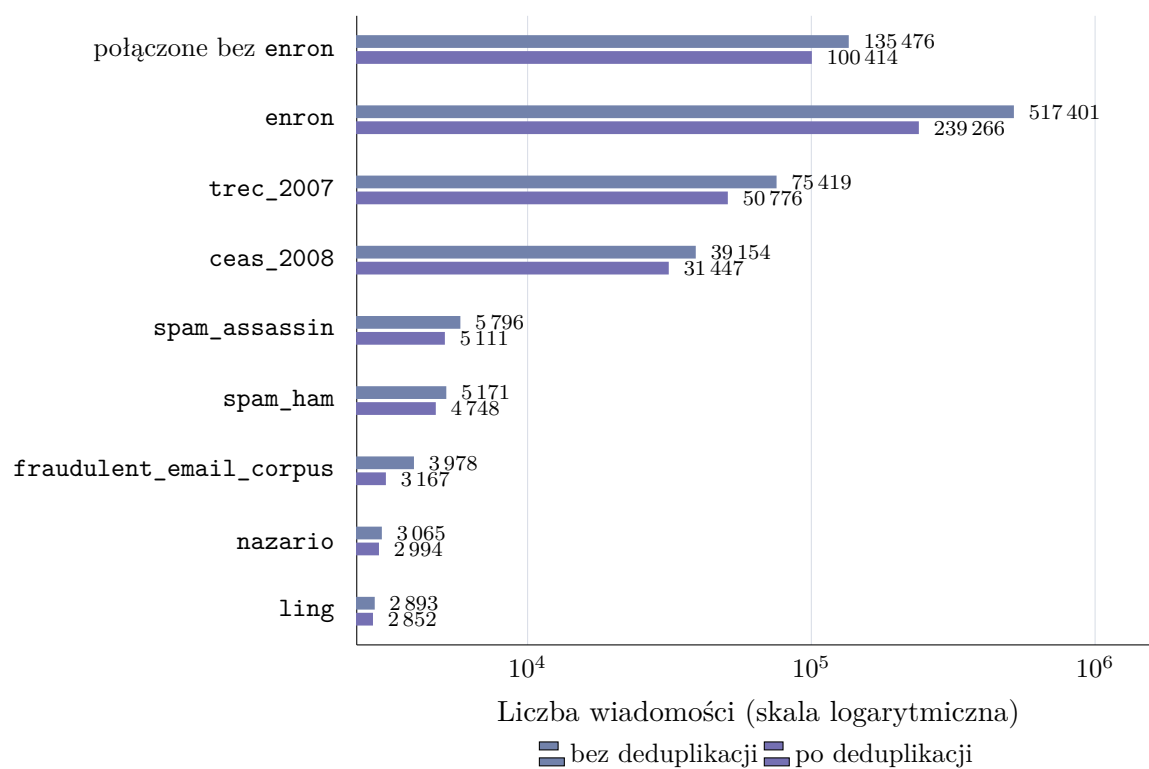
Deduplikacja ogranicza ryzyko przecieku informacji między zbiorem treningowym a testowym. Bez tego modele językowe o dużej pojemności mogłyby zapamiętywać powtarzające się wiadomości zamiast uczyć się ogólnych reguł klasyfikacji.

W pracy zastosowano kilkuwarstwową metodę wykrywania duplikatów, opartą na porównaniu tematu, treści i etykiety wiadomości. Przed porównaniem tekst normalizowano: usuwano białe znaki, tagi i encje HTML, adresy URL, e-maile oraz znaki niealfanumeryczne. Zastosowano również konwersję na małe litery oraz normalizację Unicode. Było to potrzebne, ponieważ publiczne zbiory e-mailowe często różnią się jedynie drobnymi szczegółami formatowania, nagłówkami lub linkami śledzącymi.

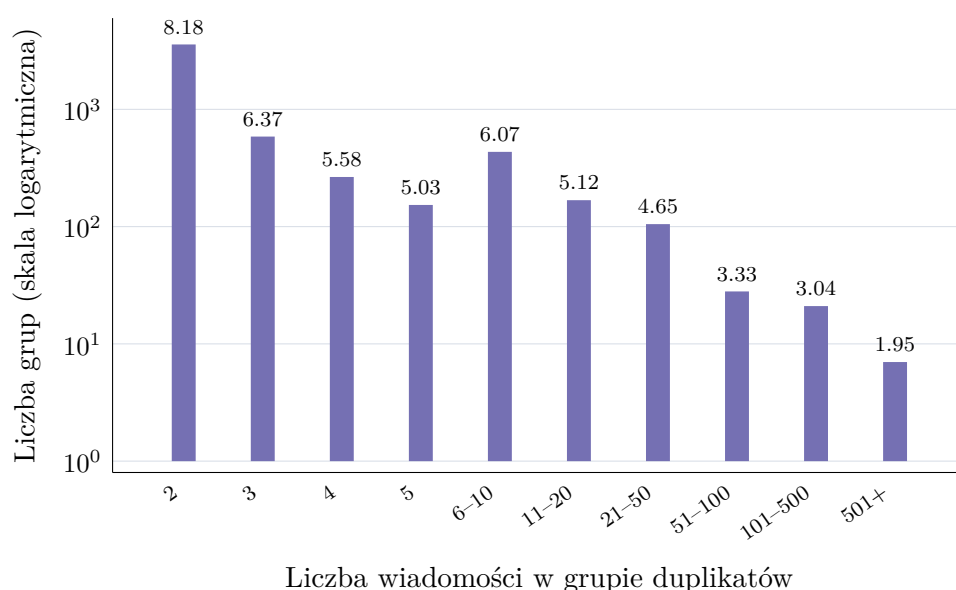
W tej części analizowano zbiór utworzony z połączenia wszystkich źródeł poza **enron**. Po normalizacji otrzymano 135 476 rekordów. Następnie odrzucono 106 wiadomości z pustym tematem i pustą treścią, pozostawiając 135 370 przykładów do deduplikacji. Deduplikacja na poziomie **high** wykryła 5344 grupy duplikatów i usunęła 34 956 nadmiarowych rekordów. Po tym etapie pozostało 100 414 wiadomości. Mediana rozmiaru grupy duplikatów wyniosła 2, a największa grupa zawierała 7832 wiadomości.

Rysunek 1.4 zestawia dwa poziomy analizy: osobne korpusy źródłowe oraz zbiór połączony ze wszystkich źródeł poza **enron**. Dzięki temu widać zarówno wpływ deduplikacji na dane używane do dalszego łączenia, jak i skalę powtórzeń w poszczególnych źródłach. Największą różnicę widać w osobno pokazanym zbiorze **enron**. Jest to korpus poczty z jednej firmy, w którym ta sama wiadomość mogła trafić do wielu odbiorców, tworząc dużą liczbę powtórzeń.

Rysunek 1.5 pokazuje, że większość grup duplikatów jest niewielka. Jest to typowe dla korpusów e-mailowych: część wiadomości występuje dokładnie lub prawie dokładnie dwa razy, natomiast bardzo duże grupy są rzadsze. Ze względu na długi ogon rozkładu większe grupy zostały połączone w przedziały. Nawet niewielkie grupy są jednak istotne, ponieważ przy losowym podziale danych mogłyby zostać rozdzielone między trening i test.



Rysunek 1.4.: Liczba rekordów przed deduplikacją oraz po deduplikacji.



Rysunek 1.5.: Rozmiary grup duplikatów po połączeniu zbiorów danych poza **enron**.

Balansowanie klas

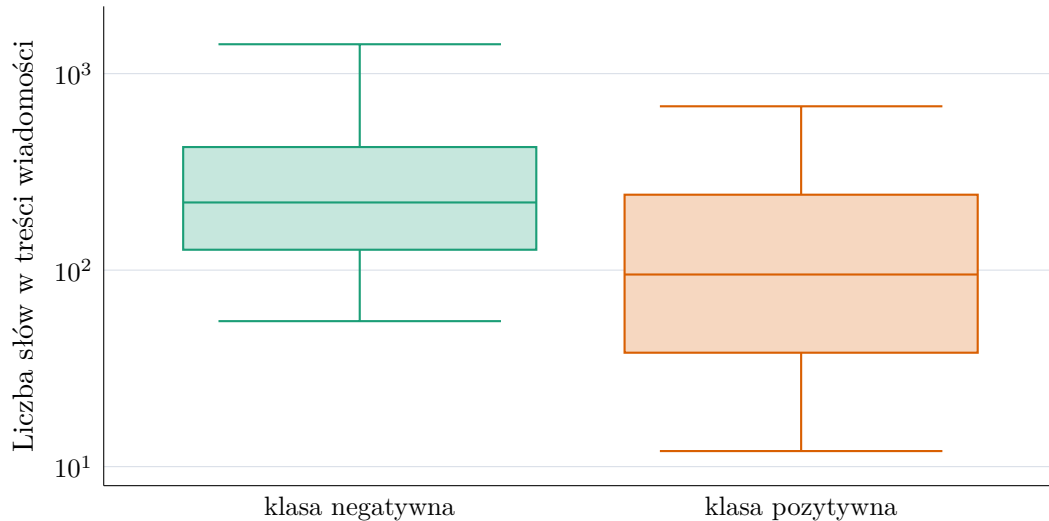
Po oczyszczeniu i deduplikacji zastosowano balansowanie klas do proporcji 50/50. Po deduplikacji zbiór utworzony ze wszystkich źródeł poza **enron** zawierał 51 418 wiadomości legalnych oraz 48 996 wiadomości niepożądanych. Balansowanie usunęło więc jedynie nadmiar przykładów z klasy negatywnej. Po balansowaniu zbiór ten zawierał 97 992 rekordy: 48 996 wiadomości pozytywnych i 48 996 negatywnych. Dzięki temu metryka dokładności staje się łatwiejsza do interpretacji, ponieważ klasy mają równy udział. Model nie jest też zachęcany do preferowania klasy większościowej. Porównanie wariantów eksperymentalnych jest stabilniejsze, ponieważ zmiany wyników nie wynikają z przypadkowego przesunięcia proporcji klas.

Balansowanie ma jednak również ograniczenia. Rzeczywisty strumień poczty użytkownika nie będzie zbalansowany. W większości środowisk spam może stanowić mały procent wiadomości, w innych znacznie większy. Dlatego wyniki na zbiorze zbalansowanym należy interpretować jako ocenę zdolności rozróżniania klas, a nie bezpośrednią symulację produkcyjnego rozkładu wiadomości.

Analiza długości wiadomości

Długość wiadomości wpływa na koszt przetwarzania przez model językowy, dobór maksymalnej liczby tokenów oraz potencjalne ucinanie wejścia. W analizowanym wariancie mediana długości treści wynosi 159 słów, natomiast 95. percentyl wynosi 1057 słów. Oznacza to,

że większość wiadomości mieści się w umiarkowanym limicie kontekstu, ale istnieje istotna grupa długich wiadomości. Jeżeli model ma ograniczony maksymalny kontekst, zbyt krótkie obcięcie treści mogłoby usuwać informacje ważne dla klasyfikacji.



Rysunek 1.6.: Rozkład długości treści wiadomości według etykiety w analizowanym wariancie danych.

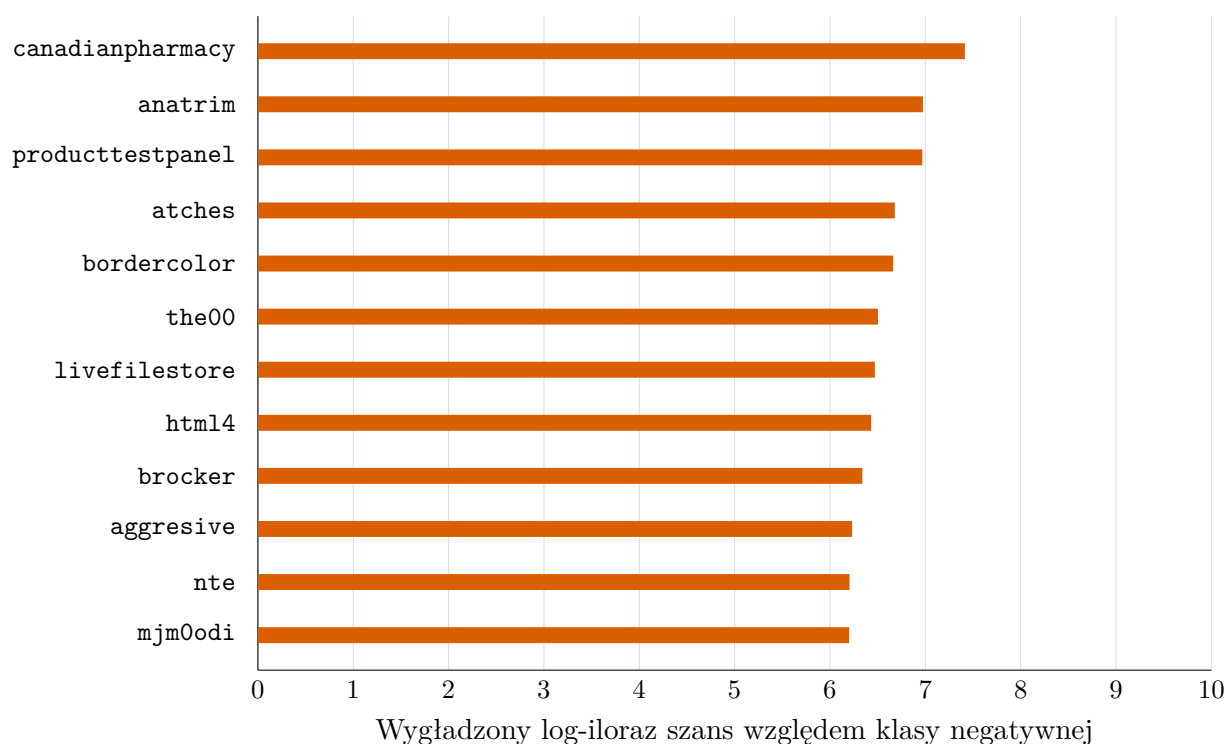
Rysunek 1.6 pokazuje różnice długości między klasami. Wiadomości legalne mają medianę 221 słów, pierwszy kwartył 127 słów, trzeci kwartył 423 słowa i 95. percentyl 1411 słów. Wiadomości spamowe są krótsze: mediana wynosi 95 słów, pierwszy kwartył 38 słów, trzeci kwartył 242 słowa, a 95. percentyl 682 słowa. Krótsza treść może wynikać z charakteru analizowanych kampanii spamowych, ale w innych zbiorach spam może być dłuższy, na przykład w wiadomościach oszukańczych typu *advance-fee fraud*[19].

Analiza według źródeł ujawnia dodatkowe różnice. W `ceas_2008` wiadomości legalne mają medianę 201 słów, a spam jedynie 40 słów. W `trec_2007` wiadomości legalne mają medianę 224 słowa, natomiast spam 147 słów. W `spam_assassin` spam ma medianę 379 słów, a w `ling` spam ma medianę 260 słów. Oznacza to, że różnica długości między klasami nie jest stabilną właściwością samego problemu, lecz zależy od źródła danych. Model może wykorzystywać tę informację, co negatywnie wpłynie na jego realną skuteczność. Jeżeli klasyfikator opierałby się głównie na długości, generalizacja do nowych typów spamu mogłaby być ograniczona.

Analiza słownictwa

Przeprowadzono także prostą analizę słów najbardziej powiązanych z klasą pozytywną i negatywną. Ranking oparto na wygładzonym ilorazie szans liczonym dla tokenów występu-

jących w temacie i treści wiadomości. Wśród terminów silnie związanych ze spamem pojawiają się między innymi `canadianpharmacy`, `anatrim`, `producttestpanel`, `atches` oraz `bordercolor`. Część z nich odpowiada klasycznym kampaniom reklamowym i farmaceutycznym, a część odzwierciedla konkretne źródła wiadomości phishingowych. Z kolei wśród terminów powiązanych z wiadomościami legalnymi pojawiają się między innymi `theweathernetwork`, `accuweather`, `reproducible`, `speakup` oraz `cbsnews`.



Rysunek 1.7.: Terminy najsilniej powiązane z klasą pozytywną według wygładzonego ilorazu szans.

Rysunek 1.7 wskazuje, że wysoka pozycja słów farmaceutycznych potwierdza, że część spamu w korpusie ma charakter reklamowy. Jednocześnie obecność bardzo specyficznych tokenów pokazuje ryzyko uczenia się artefaktów zbioru. Duży model językowy może poprawnie klasyfikować takie wiadomości, ale dobra skuteczność na korpusie zawierającym powtarzalne słowa kampanii nie musi automatycznie oznaczać odporności na nowy, bardziej wyrafinowany phishing.

1.2.3. Testowane sposoby dostrajania modelu

W eksperymentach porównano cztery sposoby dostrojenia modelu do klasyfikacji wiadomości. Wszystkie warianty korzystały z tego samego modelu bazowego `Qwen/Qwen3-0.6B` oraz adapterów LoRA, ale inaczej zapisywały zadanie i inaczej przygotowywały przykłady treningowe.

1. **Klasyfikacja sekwencji** - Model ładowano przez `AutoModelForSequenceClassification` z biblioteki `transformers`. Treść wiadomości była tokenizowana bez szablonu czatu, a kolumna `label` trafiała do trenera jako etykieta. Ten wariant stanowił punkt odniesienia dla klasycznej klasyfikacji tekstu.
2. **Następny token** - Klasyfikację sprowadzono do przewidywania następnego tokenu. Dla każdej wiadomości budowano instrukcję w formacie czatu, w której model miał wybrać jedną z dwóch etykiet. W ewaluacji model nie generował tekstu. Porównywano prawdopodobieństwa tokenów odpowiadających klasom `ham` i `spam` w pozycji następnego tokenu po instrukcji.
3. **Generowanie odpowiedzi** - Model trenowano tak, aby po instrukcji klasyfikacyjnej wygenerował krótką odpowiedź tekstową: `ham` albo `spam`. Podczas ewaluacji odpowiedź miała ograniczoną liczbę tokenów, a wynik wyznaczał parser szukający etykiety w wygenerowanym tekście.
4. **Odpowiedź strukturyzowana** - Ten wariant rozwijał poprzedni przez narzucenie formatu odpowiedzi. Model trenowano na krótkim uzupełnieniu w postaci obiektu JSON z polem `label` o wartości `spam` albo `ham`. Podczas ewaluacji parser w pierwszej kolejności szukał obiektu JSON, a dopiero potem prostszego wystąpienia pary `label: spam` lub `label: ham`.

We wszystkich wariantach liczba trenowanych parametrów była ograniczona przez adaptery LoRA. W dalszej części rozdziału najpierw przeanalizowano dobór parametrów dla metody następnego tokenu. Wybrane ustawienia stały się następnie punktem odniesienia dla pozostałych sposobów fine-tuningu.

1.2.4. Ocena modelu bazowego bez dostrajania

Przed rozpoczęciem właściwych eksperymentów sprawdzono, jak z zadaniem radzi sobie sam model `Qwen/Qwen3-0.6B`, bez aktualizacji wag i bez adapterów LoRA. Celem tego pomiaru było ustalenie, czy model bazowy ma już wystarczające właściwości zero-shot, czy dopiero dostrajanie pozwala wykorzystać go jako klasyfikator. Ewaluację wykonano na tych samych czterech zbiorach, które wykorzystano później do oceny punktów kontrolnych. Z każdego zbioru wybrano po 1000 przykładów.

Sprawdzono dwa sposoby odczytywania decyzji modelu. W pierwszym model generował krótką odpowiedź tekstową. Żeby nie zaniżyć wyniku modelu bazowego przez zbyt ogólną instrukcję, w wariancie generowania i parsowania zastosowano prompt przedstawiony na listingu 1.1.

Kod źródłowy 1.1: Prompt wykorzystany do oceny modelu bazowego w wariancie generowania i parsowania.

```

1 You are an email security classifier.
2 Classify the email as exactly one of two labels.
3
4 Definitions:
5 - spam: unsolicited advertising, scam, phishing, fraud, suspicious offer,
   malware, or otherwise unwanted email.
6 - ham: legitimate non-spam email.
7
8 Return only one lowercase word: spam or ham.
9
10 Email:
11 {email}

```

W tym wariancie zastosowano możliwie tolerancyjny parser. Oprócz odpowiedzi **ham** i **spam** rozpoznawał on także obiekt JSON, zapis typu `label: spam`, krótkie odpowiedzi opisowe oraz zaprzeczenia, na przykład `not spam`. Drugi wariant nie korzystał z generowania tekstu. Zamiast tego porównywano prawdopodobieństwa następnego tokenu **ham** oraz **spam**. Pozwoliło to oddzielić problem formatu odpowiedzi od samej preferencji modelu przy wyborze etykiety.

Zbiór	Wariant	Accuracy	F1	Recall	Specificity	Balanced acc.	Spam pred.
train_subset	generowanie	50,0	0,0	0,0	100,0	50,0	0,0
train_subset	następny token	50,0	0,0	0,0	100,0	50,0	0,0
enron	generowanie	100,0	0,0	0,0	100,0	50,0	0,0
enron	następny token	100,0	0,0	0,0	100,0	50,0	0,0
fraudulent	generowanie	0,0	0,0	0,0	0,0	0,0	0,0
fraudulent	następny token	0,0	0,0	0,0	0,0	0,0	0,0
spam_ham	generowanie	71,6	0,0	0,0	100,0	50,0	0,0
spam_ham	następny token	71,6	0,0	0,0	100,0	50,0	0,0

Tabela 1.5.: Wyniki modelu bazowego bez dostrajania. Wartości podano w procentach.

Wyniki w tabeli 1.5 pokazują, że oba warianty dały ten sam rezultat. Model w każdym przypadku wskazywał klasę **ham**, co widać po zerowym odsetku predykcji klasy **spam**. Nie wynikało to z błędu parsera, ponieważ w wariancie generacyjnym wszystkie odpowiedzi zostały poprawnie odczytane.

Na zbiorze **enron** model uzyskał 100%, ponieważ wszystkie wiadomości w tej próbkę należały do klasy **ham**. Podobnie wynik 71,6% na zbiorze **spam_ham** odpowiadał udziałowi wiadomości poprawnych w próbkę, a nie rzeczywistej zdolności wykrywania spamu. Znacznie lepiej pokazują to metryki związane z klasą pozytywną: *recall* oraz F1 dla spamu wyniosły 0% na każdym zbiorze zawierającym wiadomości niepożądane. Na zbiorze **fraudulent_email_corpus**, który składa się wyłącznie z wiadomości oszukańczych, model nie wskazał poprawnie ani jednego przykładu.

Wyniki modelu bazowego pokazują więc, że samo sformułowanie zadania w postaci instrukcji nie wystarczało w tej konfiguracji.

1.2.5. Dobór parametrów treningu

Po przygotowaniu danych i przetestowaniu modelu bazowego dobrano parametry fine-tuningu. Eksperymenty przeprowadzono dla metody opartej na przewidywaniu następnego tokenu. Model porównywał prawdopodobieństwa dwóch tokenów odpowiedzi: **ham** oraz **spam**, a klasę końcową wybierał na podstawie większego prawdopodobieństwa.

Eksperymenty przeprowadzono dla modelu **Qwen/Qwen3-0.6B**. Do dostrajania zastosowano adapter **LoRA**. We wszystkich konfiguracjach użyto tego samego **seed**'a równego 67, aby zachować powtarzalność podziału danych i próbkowania.

Przestrzeń hiperparametrów

Przeanalizowano 16 konfiguracji. Zmieniane parametry obejmowały maksymalną długość kontekstu, współczynnik uczenia¹, pojemność adaptera **LoRA** oraz dropout. Zmiany parametrów można podzielić na trzy grupy. Pierwsza grupa eksperymentów badała wpływ długości kontekstu i współczynnika uczenia. Druga grupa porównywała różne parametry **LoRA**, a trzecia zmieniała współczynnik dropout. Większej liczby kombinacji parametrów nie przetestowano ze względu na ograniczone zasoby. Tabela 1.6 przedstawia zakres konfiguracji objętych analizą punktów kontrolnych.

Ewaluacja pośrednich punktów kontrolnych

Oceniono zarówno końcowe adaptory, jak i pośrednie punkty kontrolne². Celem było znalezienie momentu, w którym model jest już dobrze dopasowany do danych treningowych, ale nie traci skuteczności na zbiorach zewnętrznych.

Dla każdej konfiguracji analizowano maksymalnie 10 punktów treningu: pierwszy punkt kontrolny, ostatni punkt kontrolny oraz punkty pośrednie rozłożone możliwie równomiernie. Ograniczało to koszt ewaluacji, ale nadal dawało obraz przebiegu uczenia. Łącznie oceniono

¹ang. *learning rate*

²ang. *checkpoint*

Konf.	Kontekst	LR	LoRA r	LoRA alpha	Dropout
K01	512	1e-06	16	32	0.05
K02	512	5e-05	16	32	0.05
K03	512	1e-04	16	32	0.05
K04	1024	1e-06	16	32	0.05
K05	1024	5e-05	16	32	0.05
K06	1024	1e-04	16	32	0.05
K07	512	1e-04	8	16	0.05
K08	512	1e-04	32	64	0.05
K09	512	1e-04	64	128	0.05
K10	1024	1e-04	8	16	0.05
K11	1024	1e-04	32	64	0.05
K12	1024	1e-04	64	128	0.05
K13	512	1e-04	16	32	0.00
K14	512	1e-04	16	32	0.40
K15	1024	1e-04	16	32	0.00
K16	1024	1e-04	16	32	0.40

Tabela 1.6.: Konfiguracje treningu objęte analizą punktów kontrolnych.

160 punktów treningu, po cztery zbiory danych dla każdego punktu. Daje to 640 zakończonych pomiarów.

Zbiory użyte do ewaluacji

Ewaluację przeprowadzono oddzielnie dla czterech zbiorów. Pierwszy zbiór, oznaczony jako `train_subset`, wydzielono z części treningowej `training_all`. Jego zadaniem było sprawdzenie, jak dobrze model radzi sobie z przykładami pochodzącymi z tej samej dystrybucji co dane treningowe. Nie jest to jednak całkowicie niezależny test generalizacji, ponieważ dane te są blisko powiązane ze zbiorem używanym do aktualizacji wag adaptera.

Poza podzbiorem `train_subset` wykorzystano również trzy kolejne, niezależne zbiory. `enron` zawiera wyłącznie wiadomości klasy `ham`, dlatego wykorzystano go do oceny błędów typu *false positive*. Zbiór `fraudulent_email_corpus` zawiera wyłącznie wiadomości oszukańcze, dlatego służy do oceny *recall* względem spamu o charakterze odmiennym od wiadomości treningowych. `spam_ham` jest natomiast zbiorem mieszanym, dlatego pozwala ocenić metryki zadania binarnego, w szczególności F1 dla klasy pozytywnej. Każdy z tych zbiorów został ograniczony do 1000 przykładów.

Metryki oceny

Wyniki analizowano za pomocą metryk *accuracy*, *precision*, *recall*, *F1*, *specificity*, *balanced accuracy*, *false positive rate* oraz *false negative rate*. Ponieważ część zbiorów zewnętrznych jest jednoklasowa, nie wszystkie metryki są jednakowo informacyjne. Dla **enron** kluczowa jest *specificity*, a dla **fraudulent_email_corpus** *recall*.

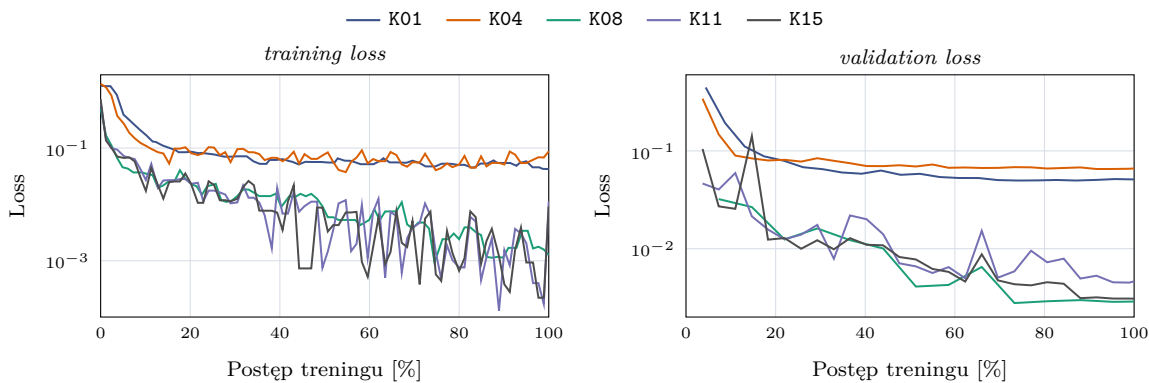
Z tego powodu do porównania konfiguracji wykorzystano następujący wzór:

$$S_{\text{ext}} = \frac{\text{specificity}_{\text{enron}} + \text{recall}_{\text{fraudulent}} + \text{F1}_{\text{spam_ham}}}{3}.$$

W ten sposób porównanie uwzględniało trzy różne typy danych zewnętrznych, a nie tylko jedną metrykę z jednego zbioru.

Statystyki przebiegu treningu

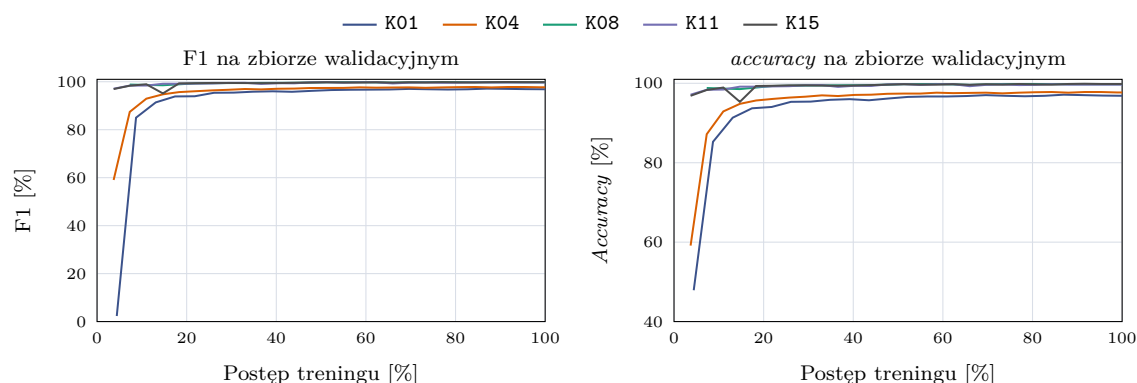
Przeanalizowano również metryki zapisywane podczas treningu. Nie były one głównym kryterium wyboru modelu, ale pomagały sprawdzić, jak szybko model dostosowuje się do danych treningowych i jak zmienia się koszt różnych konfiguracji. Do porównania wybrano konfiguracje K01, K04, K08, K11 oraz K15. K01 i K04 reprezentują warianty z niskim współczynnikiem uczenia, K11 ma podobną pojemność adaptera przy kontekście 1024 tokenów, a K15 odpowiada wariantowi bez dropout przy dłuższym kontekście.



Rysunek 1.8.: Przebieg funkcji straty dla wybranych konfiguracji treningu. Wykres *training loss* został wygładzony medianą kroczącą, aby ograniczyć szum wynikający z pomiaru na kolejnych batchach.

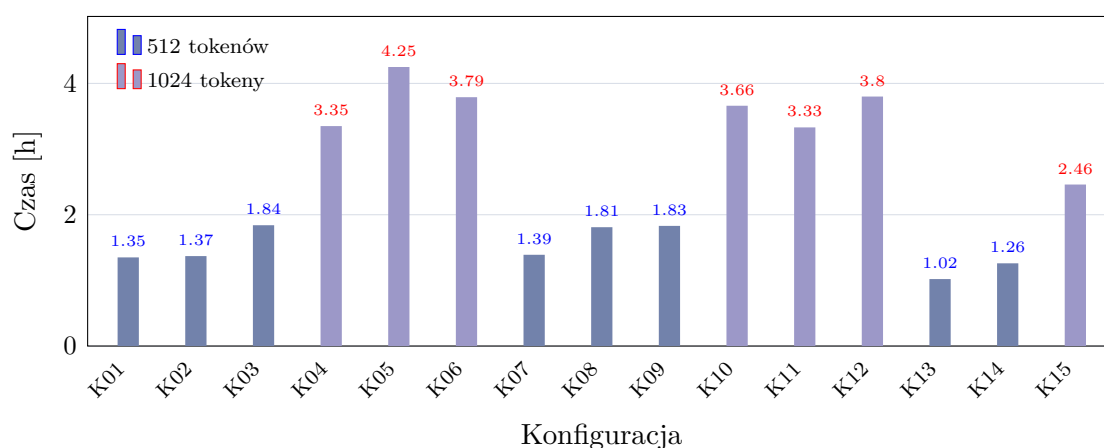
Na rysunku 1.8 widać, że konfiguracje z niskim współczynnikiem uczenia wolniej zmniejszały wartość funkcji straty. Dotyczy to przede wszystkim K01 i K04, czyli tych samych wariantów, które później uzyskały słabsze wyniki na zbiorach zewnętrznych. Dla K08, K11

i K15 strata walidacyjna szybko spadała do niskiego poziomu. Sam przebieg straty nie wystarczył jednak do wyboru konfiguracji, ponieważ po początkowej poprawie dalszy trening nie przekładał się jednoznacznie na lepszy wynik zewnętrzny.



Rysunek 1.9.: Przebieg F1 oraz *accuracy* na zbiorze walidacyjnym dla wybranych konfiguracji treningu.

Rysunek 1.9 pokazuje podobne zjawisko. F1 i *accuracy* na zbiorze walidacyjnym bardzo szybko osiągały wartości bliskie 100%. Model dobrze dopasowywał się więc do danych walidacyjnych, ale same metryki walidacyjne słabo rozdzielały najlepsze konfiguracje. O wyborze parametrów decydowały przede wszystkim wyniki na *enron*, *fraudulent_email_corpus* i *spam_ham*.



Rysunek 1.10.: Czas treningu poszczególnych konfiguracji. Kolor słupka oznacza maksymalną długość kontekstu. Porównanie obejmuje 15 treningów, dla których zapisano metrykę *train_runtime*.

Rysunek 1.10 pokazuje, że największy wpływ na czas treningu miała maksymalna długość kontekstu. Konfiguracje z limitem 512 tokenów trenowały się od około 1,02 do 1,84 godziny, natomiast warianty z limitem 1024 tokenów od około 2,46 do 4,25 godziny. Średnio oznaczało to wzrost z 1,48 do 3,52 godziny oraz spadek przepustowości z około 16,0 do 6,6 próbki na sekundę. Pozostałe parametry także wpływały na czas wykonania, ale w mniejszym i mniej regularnym stopniu. Większy rząd LoRA nie powodował jednoznacznego wzrostu czasu treningu w każdej grupie, a różnice między konfiguracjami o tym samym kontekście były mniejsze niż różnica między samymi długościami kontekstu. Dłuższy kontekst był więc wyraźnie droższy, a jak pokazano w dalszej części pracy, nie dawał przewagi uzasadniającej taki koszt.

Ranking konfiguracji

Tabela 1.7 przedstawia ranking najlepszych punktów kontrolnych dla poszczególnych konfiguracji. Dla każdej konfiguracji wybrano punkt kontrolny o najwyższym wyniku zewnętrznym. W tabeli uwzględniono pozycję w rankingu, identyfikator konfiguracji, numer wybranego punktu kontrolnego, wynik zewnętrzny S_{ext} , F1 na **train_subset**, F1 na **spam_ham** oraz różnicę między F1 na **train_subset** a wynikiem zewnętrznym.

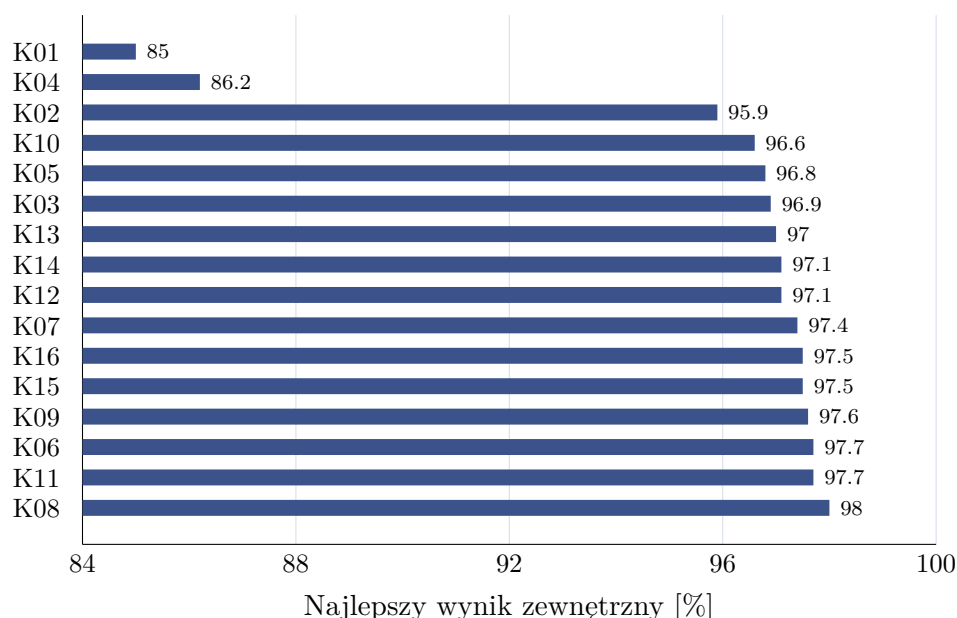
Najlepszy rezultat uzyskała konfiguracja K08, czyli wariant z kontekstem 512 tokenów, współczynnikiem uczenia 10^{-4} , rzędem LoRA równym 32, parametrem alpha równym 64 oraz dropout równym 0,05. Najlepszy punkt kontrolny tej konfiguracji miał numer 7 i osiągnął wynik zewnętrzny 98,0%.

Miejsce	Konf.	Nr punktu kontrolnego	Wynik zewn. [%]	train_subset F1 [%]	spam_ham F1 [%]	Luka [p.p.]
1	K08	7	98.0	99.9	97.2	1.9
2	K11	7	97.7	99.9	97.0	2.2
3	K06	8	97.7	99.9	96.7	2.2
4	K09	7	97.6	99.9	96.3	2.3
5	K15	8	97.5	99.8	96.2	2.3
6	K16	8	97.5	99.9	95.5	2.4
7	K07	7	97.4	99.8	96.2	2.4
8	K12	8	97.1	99.7	94.3	2.6
9	K14	5	97.1	100.0	96.6	2.9
10	K13	7	97.0	99.8	95.4	2.8

Tabela 1.7.: Ranking najlepszych punktów kontrolnych dla konfiguracji metody klasyfikacji następnego tokenu. Wynik zewnętrzny jest średnią z *specificity* na **enron**, *recall* na **fraudulent_email_corpus** oraz F1 na **spam_ham**.

Rysunek 1.11 pokazuje, że większość konfiguracji ze współczynnikiem uczenia 10^{-4} osiąga bardzo zbliżone wyniki zewnętrzne. Różnice między najlepszymi wariantami mieszczą się w kilku dziesiątych punktu procentowego. Wyraźnie słabsze są natomiast warianty K01 i K04, które wykorzystywały współczynnik uczenia 10^{-6} . Osiągnęły one wyniki zewnętrzne odpowiednio 85,0% i 86,2%, a więc znacząco niższe od pozostałych konfiguracji. Sugeruje to,

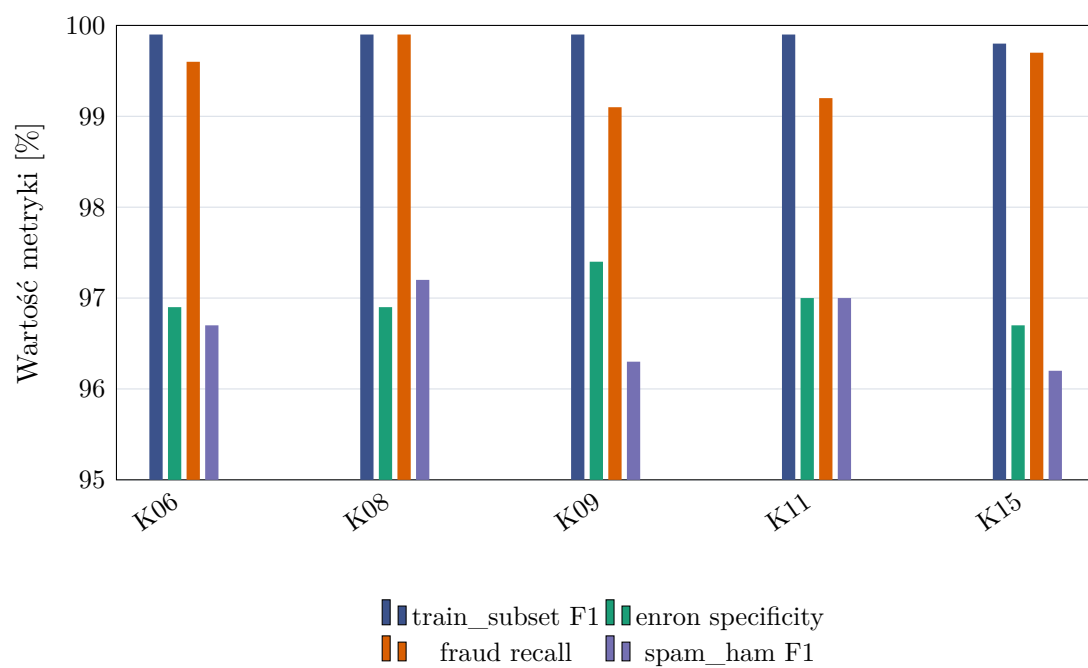
że przy przyjętej liczbie epok i rozmiarze danych tak niski współczynnik uczenia nie pozwalał dostatecznie zaadaptować modelu do zadania.



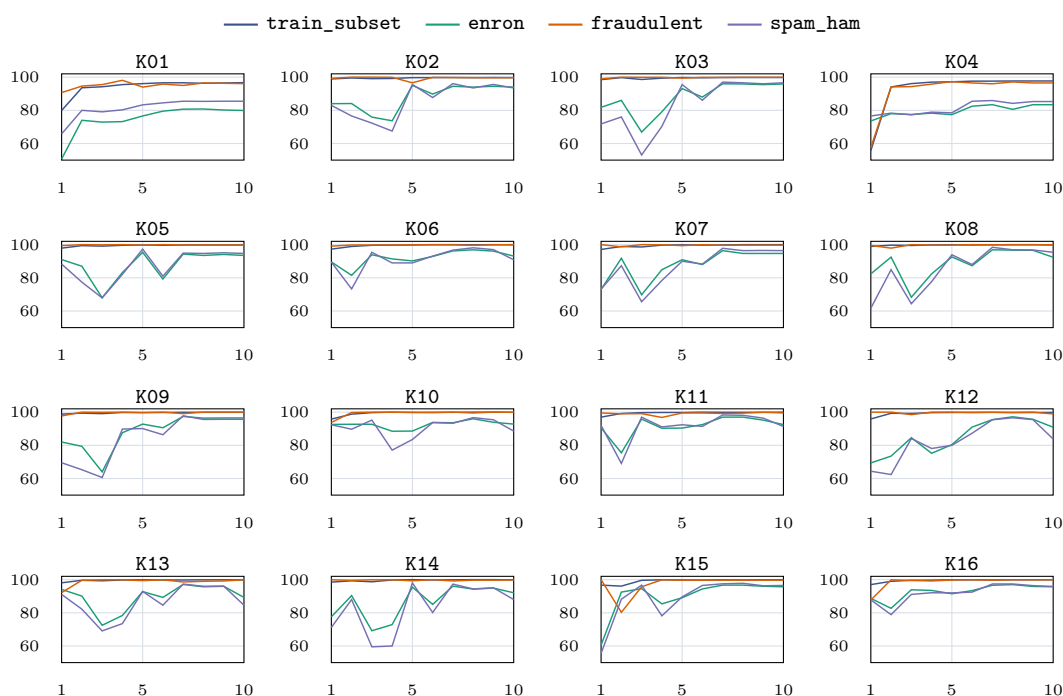
Rysunek 1.11.: Najlepszy wynik zewnętrzny uzyskany przez każdą analizowaną konfigurację metody klasyfikacji następnego tokenu.

Rysunek 1.12 przedstawia szczegóły dla pięciu najlepszych konfiguracji. Wszystkie osiągają bardzo wysokie F1 na `train_subset`, bliskie 100%. Najważniejsze różnice widoczne są jednak na zbiorach zewnętrznych. Konfiguracja K08 zachowuje wysokie *specificity* na `enron`, bardzo wysoki *recall* na `fraudulent_email_corpus` oraz najwyższy F1 na `spam_ham` wśród porównywanych wariantów. Wynik ten wskazuje na stabilność tej konfiguracji przy zmianie źródła danych.

Uzupełniając przeanalizowano również przebieg *accuracy* dla wszystkich analizowanych konfiguracji oraz wszystkich zbiorów ewaluacyjnych. Rysunek 1.13 ma charakter informacyjny i pokazuje, jak zmieniała się skuteczność w kolejnych ocenianych punktach kontrolnych. Wnioski dotyczące wyboru parametrów oparto jednak na opisanym wcześniej wyniku zewnętrznym oraz analizie luki generalizacji.



Rysunek 1.12.: Porównanie metryk dla pięciu najlepszych konfiguracji według wyniku zewnętrznego.



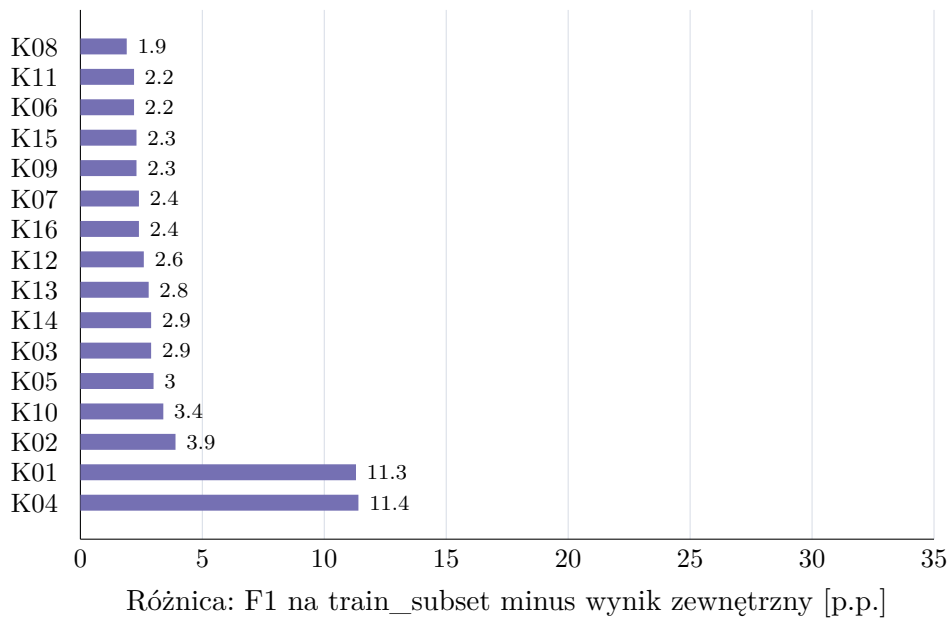
Rysunek 1.13.: Przebieg *accuracy* dla analizowanych konfiguracji. Każdy panel odpowiada jednej konfiguracji, a linie przedstawiają osobne zbiory ewaluacyjne.

Analiza przeuczenia

Porównanie wyników na `train_subset` i zbiorach zewnętrznych pozwala ocenić ryzyko przeuczenia. Sam wynik na podzbiorze treningowym jest niewystarczający, ponieważ większość konfiguracji osiąga tam F1 bliskie 100%. Gdyby wybór opierał się tylko na tej metryce, trudno byłoby odróżnić modele, które rzeczywiście dobrze generalizują, od modeli nadmierne dopasowanych do charakterystyki zbioru treningowego.

Rysunek 1.14 pokazuje lukę między F1 na `train_subset` a wynikiem zewnętrznym. Najmniejszą lukę spośród analizowanych konfiguracji ma K08. Różnica wynosi około 1,9 punktu procentowego. Dla większości dobrych konfiguracji luka mieści się w zakresie od około 2 do 3 punktów procentowych. Wyjątkami są K01 i K04, dla których luka wynosi ponad 11 punktów procentowych. Oznacza to, że konfiguracje z najniższym współczynnikiem uczenia znacznie gorzej przenoszą skuteczność ze zbioru podobnego do treningowego na dane o innej charakterystyce.

Analiza przebiegu punktów kontrolnych wskazuje również, że wybór końcowego adaptera nie zawsze jest optymalny. Rysunek 1.15 przedstawia zmianę wyniku zewnętrznego w kolejnych ocenianych punktach kontrolnych dla pięciu najlepszych konfiguracji. We wszystkich przypadkach najlepszy wynik pojawia się przed końcem treningu. Dla K08 najlepszy wynik



Rysunek 1.14.: Luka generalizacji między F1 na `train_subset` a wynikiem zewnętrznym dla najlepszych punktów kontrolnych poszczególnych konfiguracji.

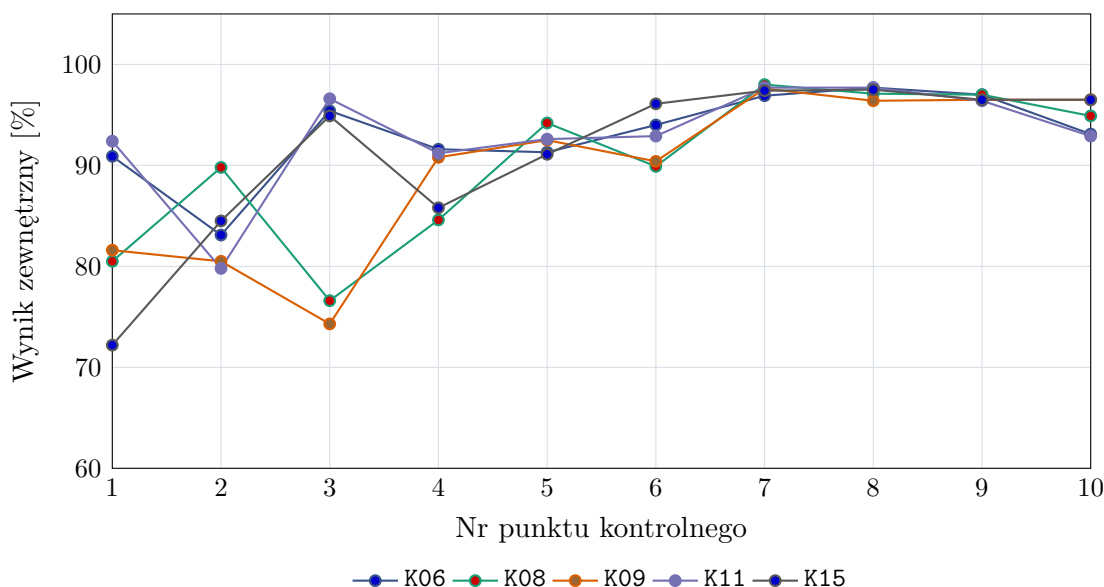
uzyskano dla punktu kontrolnego nr 7, mimo że dalsze punkty kontrolne nadal osiągają wysokie wartości. Sugeruje to, że po tym etapie dalsze dopasowywanie do zbioru treningowego zaczyna pogarszać wynik na danych zewnętrznych. Podobny przebieg widać w pozostałych czterech najlepszych konfiguracjach.

Wybór najlepszej konfiguracji

Na podstawie wyników jako najlepszą konfigurację metody klasyfikacji następnego tokenu wybrano K08. Konfiguracja ta osiągnęła najwyższy wynik zewnętrzny spośród ocenionych wariantów. Miała również najmniejszą lukę generalizacji, co ogranicza ryzyko wyboru modelu nadmiernie dopasowanego do danych treningowych. Wyniki na zbiorach zewnętrznych wskazują również, że model zachowywał wysoką wykrywalność spamu, a jednocześnie utrzymywał niski odsetek fałszywych pozytywów na wiadomościach typu `ham`.

Wybrane parametry to maksymalna długość kontekstu 512 tokenów, współczynnik uczenia 10^{-4} , LoRA $r = 32$, LoRA $\alpha = 64$ oraz dropout równy 0,05. Najlepszy oceniony punkt kontrolny miał numer 7. Długość kontekstu 512 tokenów ogranicza także koszt treningu i inferencji względem wariantów 1024-tokenowych. Wyniki wskazują, że w tej serii eksperymentów dłuższy kontekst nie był konieczny do uzyskania najlepszej generalizacji.

W obrębie analizowanej metody konfiguracja K08 stanowi najlepszy kompromis między skutecznością na danych podobnych do treningowych, odpornością na zmianę źródła da-

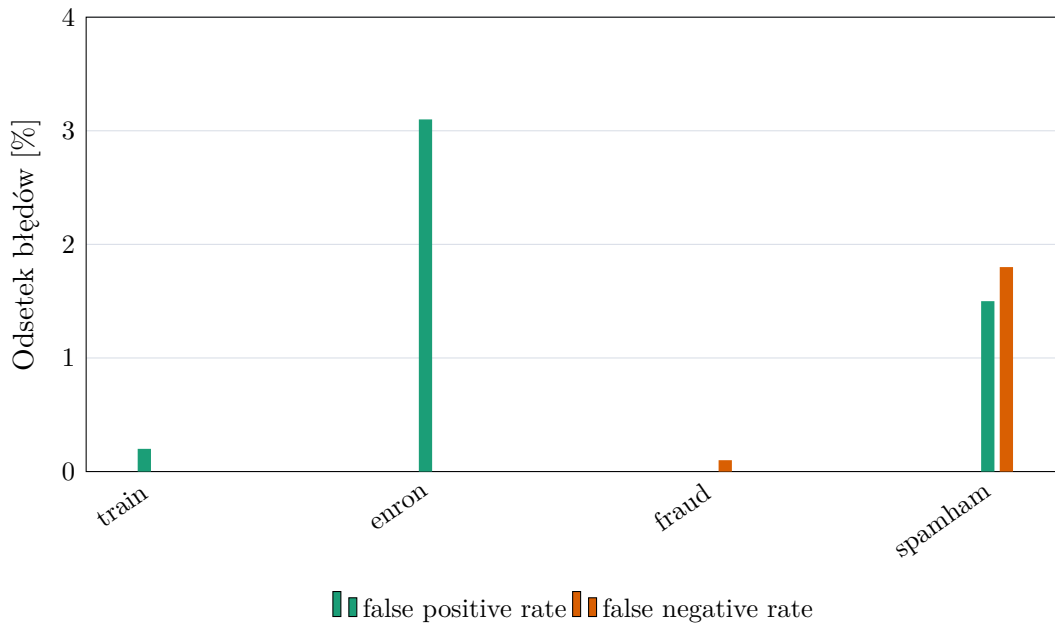


Rysunek 1.15.: Przebieg wyniku zewnętrznego dla pięciu najlepszych konfiguracji w kolejnych ocenianych punktach kontrolnych.

nych oraz kosztem obliczeniowym. Wysoki wynik na `fraudulent_email_corpus` wskazuje, że model poprawnie rozpoznaje wiadomości oszukańcze o charakterystyce innej niż część wiadomości treningowych. Wynik na `enron` pokazuje natomiast, że model nie osiąga wysokiej wykrywalności spamu kosztem nadmiernej liczby fałszywych alarmów. Z tego powodu konfiguracja K08 została przyjęta jako reprezentatywny wariant metody klasyfikacji następnego tokenu.

Profil błędów wybranej konfiguracji

Dla wybranej konfiguracji K08 przeanalizowano również profil błędów. Rysunek 1.16 zestawia odsetek decyzji typu *false positive* i *false negative* na poszczególnych zbiorach. Na `enron` odsetek fałszywych alarmów wynosi około 3,1%, co oznacza, że model relatywnie rzadko oznacza legalną korespondencję jako spam. Na `fraudulent_email_corpus` odsetek decyzji typu *false negative* wynosi około 0,1%, a więc model prawie wszystkie wiadomości oszukańcze rozpoznaje jako klasę pozytywną. Na mieszanym zbiorze `spam_ham` *false positive* i *false negative* są zbliżone i nie przekraczają 2%. Model nie osiąga więc wysokiej skuteczności kosztem skrajnego przesunięcia w stronę jednej klasy.



Rysunek 1.16.: Odsetki błędów typu *false positive* i *false negative* dla wybranego punktu kontrolnego konfiguracji K08.

Wnioski z eksperymentu

Analiza pokazała, że wynik na `train_subset` nie wystarczał do wyboru najlepszej konfiguracji. Najlepsze warianty miały tam bardzo wysokie F1, więc różnice między nimi było widać dopiero na zbiorach zewnętrznych: `enron`, `fraudulent_email_corpus` i `spam_ham`.

Najgorzej wypadły warianty K01 i K04, trenowane ze współczynnikiem uczenia 10^{-6} . Uzywały niższe wyniki na zbiorach zewnętrznych i miały największe luki generalizacji. Lepsze okazały się konfiguracje z 10^{-4} . Wśród nich najlepiej wypadł wariant z $r = 32$ i $\alpha = 64$, co wskazuje, że bardzo mały adapter LoRA był zbyt ograniczony, ale dalsze zwiększanie jego pojemności nie dawało już wyraźnej poprawy.

Nie było też potrzeby stosowania kontekstu 1024-tokenowego. Najlepszy wariant korzystał z limitu 512 tokenów, więc był tańszy obliczeniowo, a jednocześnie zachowywał wystarczająco dużo informacji do klasyfikacji.

Wyniki punktów kontrolnych pokazały, że ostatni zapisany adapter nie zawsze był najlepszy. W kilku konfiguracjach lepsze wyniki na zbiorach zewnętrznych pojawiły się wcześniej. Dlatego przy wyborze modelu warto sprawdzać nie tylko metryki walidacyjne, ale też okresowo oceniać model na danych z innych źródeł.

Klasyfikacja przez przewidywanie następnego tokenu sprawdziła się w tym zadaniu. Po dostrojeniu model dobrze wykrywał wiadomości oszukańcze i utrzymywał niski odsetek fał-

szywych alarmów. Najlepszym wariantem był K08; w dalszej analizie traktowano go więc jako właściwy punkt odniesienia dla metody zwracającej etykietę `ham/spam`.

1.2.6. Porównanie metod dostrajania

Po wybraniu najlepszej konfiguracji dla wariantu przewidywania następnego tokenu porównano pozostałe sposoby dostrajania modelu. W tym etapie nie powtarzano pełnego przeszukiwania hiperparametrów. Sprawdzano raczej, czy przy zbliżonych ustawieniach treningu znaczenie ma sama postać zadania: klasyfikacja sekwencji, generowanie etykiety, ocena następnego tokenu albo generowanie odpowiedzi strukturyzowanej. Dla metod 01 i 02 oceniono cztery najlepsze konfiguracje wybrane na podstawie wcześniejszego eksperymentu. Metoda 03 obejmuje pełny ranking 16 konfiguracji opisany wcześniej. Dla metody 04 dostępne były wyniki dwóch konfiguracji, dlatego jej ocenę potraktowano jako częściową.

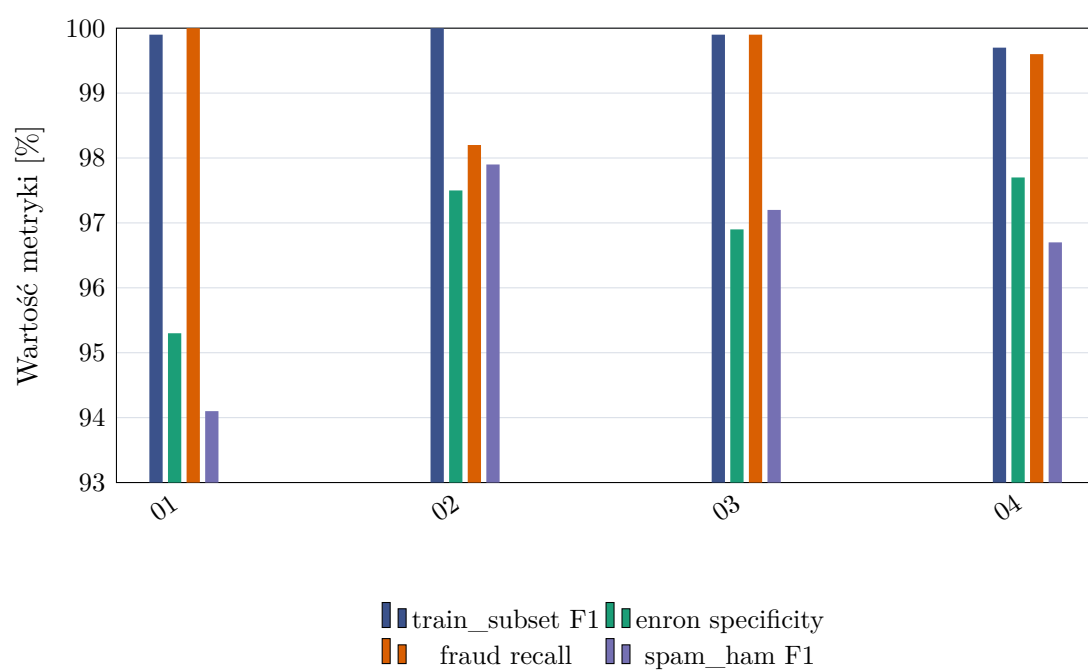
Przy wyborze metody liczył się nie tylko wynik zbiorczy, ale też sposób działania modelu podczas inferencji. Klasyfikacja sekwencji zwraca decyzję bez generowania tekstu i bez parsera, ale wymaga osobnej głowicy klasyfikacyjnej. Generowanie odpowiedzi jest proste koncepcyjnie, jednak wynik zależy od tego, czy parser poprawnie odczyta tekst zwrócony przez model. Metoda następnego tokenu porównuje bezpośrednio prawdopodobieństwa etykiet `ham` i `spam`, więc nie wprowadza osobnego problemu parsowania. Odpowiedź strukturyzowana ogranicza swobodę formatu przez wymuszenie pola `label`, ale nadal pozostaje wariantem opartym na generowaniu tekstu.

Metoda	Konf.	Nr punktu	train_subset F1 [%]	enron spec. [%]	fraudulent recall [%]	spam_ham F1 [%]	S_{ext} [%]
01	K06	10	99.9	95.3	100.0	94.1	96.5
02	K08	7	100.0	97.5	98.2	97.9	97.9
03	K08	7	99.9	96.9	99.9	97.2	98.0
04*	K08	7	99.7	97.7	99.6	96.7	98.0

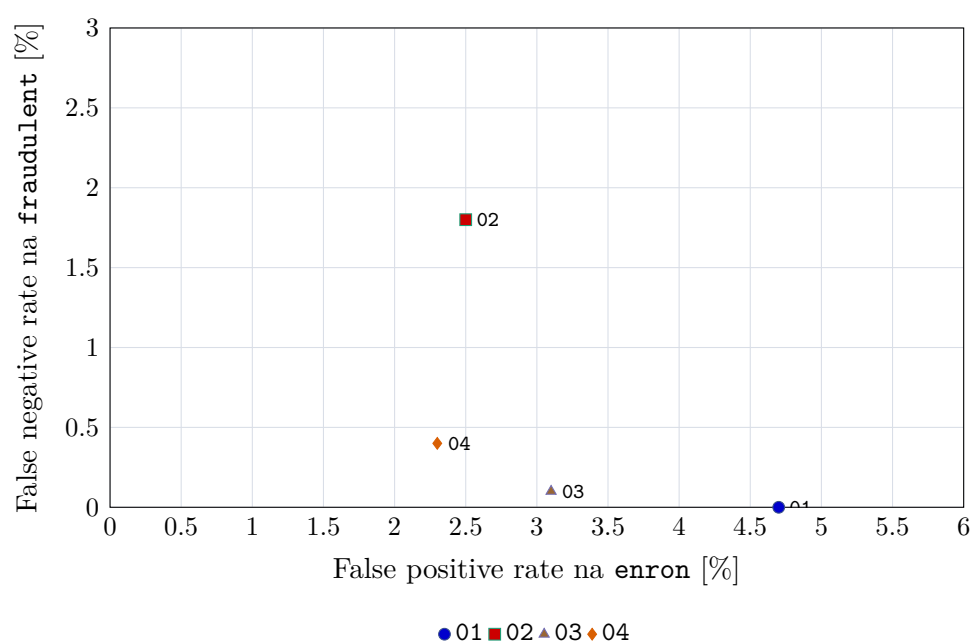
Tabela 1.8.: Najlepsze punkty kontrolne uzyskane dla porównywanych metod. Gwiazdka przy metodzie 04 oznacza wynik częściowy, obejmujący dwie ocenione konfiguracje.

Najlepsze warianty metod opartych na modelu językowym uzyskały bardzo zbliżone wyniki zewnętrzne. Najwyższą wartość S_{ext} osiągnęła metoda 03, ale po zaokrągleniu metoda 04 ma ten sam wynik, a metoda 02 jest niższa o około 0,1 punktu procentowego. Różnice nie wskazują więc na jednoznaczną dominację jednej metody we wszystkich warunkach ewaluacji.

Różnice między metodami widać wyraźniej dopiero po rozbiciu wyniku na poszczególne zbiory ewaluacyjne. Klasyfikacja sekwencji, czyli metoda 01, osiąga bardzo wysoki wynik na `train_subset` i pełny *recall* na `fraudulent_email_corpus`, ale wypada słabiej na `enron` oraz `spam_ham`. Taki profil oznacza, że metoda dobrze uczy się rozpoznawania wiadomości



Rysunek 1.17.: Porównanie najlepszych wariantów metod na czterech zbiorach ewaluacyjnych. Dla `train_subset` i `spam_ham` pokazano F1, dla `enron specificity`, a dla `fraudulent_email_corpus recall`.



Rysunek 1.18.: Porównanie błędów typu *false positive* na zbiorze **enron** oraz *false negative* na zbiorze **fraudulent_email_corpus** dla najlepszych wariantów metod. Punkty położone bliżej lewego dolnego rogu mają korzystniejszy profil błędów.

oszukańczych, lecz częściej niż pozostałe warianty oznacza poprawną korespondencję jako spam i gorzej radzi sobie na zbiorze mieszanym.

Metody 02, 03 i 04 są znacznie bliżej siebie. Wariant generowania i parsowania odpowiedzi ma najwyższy F1 na **spam_ham**, ale słabszy *recall* na zbiorze wiadomości oszukańczych. Dla dwóch ocenionych konfiguracji metoda 04 lokuje się blisko najlepszych wariantów: osiąga najwyższą *specificity* na **enron** i bardzo wysoki *recall*, lecz jej F1 na **spam_ham** jest niższe niż w metodach 02 i 03. Wynik wskazuje na potencjał tej metody, ale nie wystarcza jeszcze do pełnego porównania z pozostałymi wariantami.

Rysunek 1.18 pokazuje tę różnicę od strony profilu błędów. Metoda 01 nie przepuszcza wiadomości oszukańczych w zbiorze **fraudulent_email_corpus**, ale robi to kosztem większego odsetka fałszywych alarmów na **enron**. Metoda 02 ma odwrotny profil: rzadziej oznacza poprawne wiadomości jako spam, lecz częściej pomija wiadomości oszukańcze. Metody 03 i 04 znajdują się bliżej lewego dolnego rogu, co oznacza bardziej zrównoważony kompromis między tymi dwoma rodzajami błędów.

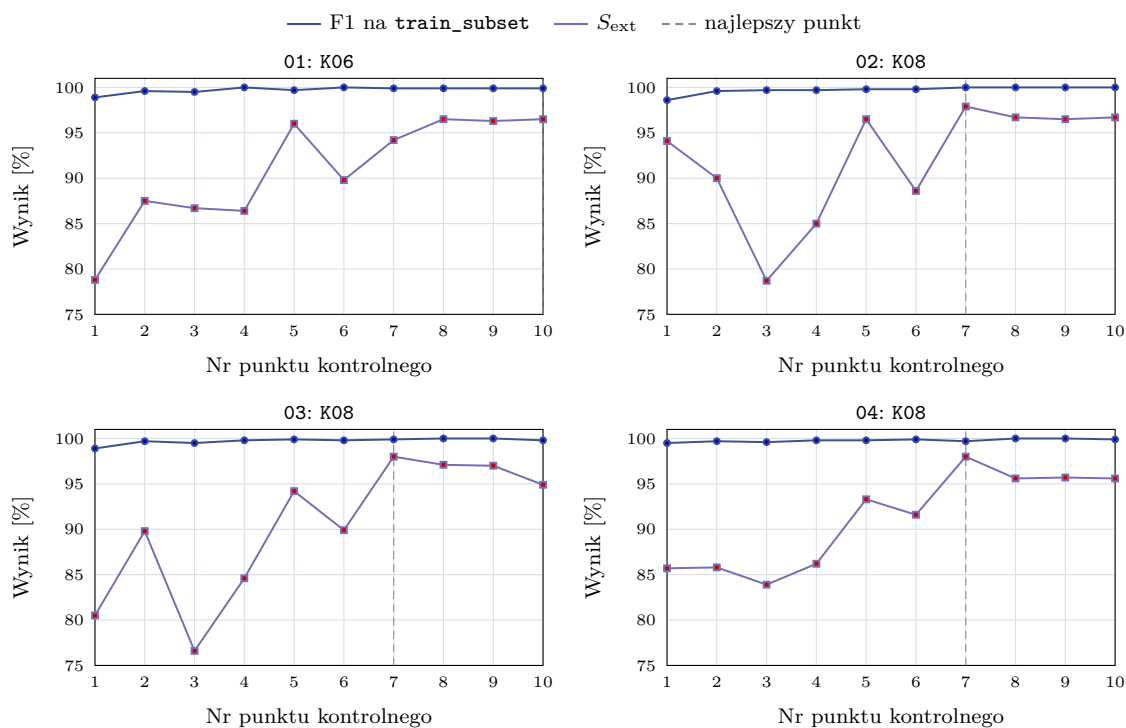
Najbardziej wyrównany profil ma metoda 03. Nie jest najlepsza na każdym pojedyńczym zbiorze, ale uzyskuje najwyższy wynik zbiorczy w wartościach niezaokrąglonych, prawie pełny *recall* na zbiorze wiadomości oszukańczych oraz wysokie F1 na **spam_ham**. Ważna jest też prostota inferencji: model nie musi generować odpowiedzi ani utrzymywać formatu tekstowego, ponieważ decyzja wynika z porównania dwóch prawdopodobieństw.

Do porównania włączono także przebieg wyników w kolejnych punktach kontrolnych. Metoda, która wcześniej osiąga dobry wynik, jest praktycznie łatwiejsza do strojenia, nawet jeśli nie musi mieć krótszego czasu ściennego treningu. Tego czasu nie porównano bezpośrednio, ponieważ pliki **summary.csv** z ewaluacji punktów kontrolnych nie zawierają pełnych metryk czasu dla wszystkich metod. Zestawiono więc liczbę ocenianych punktów kontrolnych oraz odpowiadający im numer kroku.

Metoda	Konf.	Najlepszy nr punktu	Krok	S_{ext} [%]	Punkty konf.
01	K06	10	final	96.5	10
02	K08	7	3000	97.9	10
03	K08	7	3000	98.0	10
04*	K08	7	3000	98.0	10

Tabela 1.9.: Moment uzyskania najlepszego wyniku zewnętrznego dla najlepszej konfiguracji każdej metody.

We wszystkich metodach F1 na **train_subset** rosło już w pierwszych ocenianych punktach kontrolnych i szybko zbliżało się do wartości bliskich 1. Wynik na danych podobnych do treningowych stabilizował się więc wcześniej niż wynik zewnętrzny. Krzywa S_{ext} była mniej regularna, zwłaszcza w pierwszej części treningu. Z tego powodu sam wynik na **train_subset** nie był wystarczającym kryterium wyboru modelu.



Rysunek 1.19.: Przebieg F1 na `train_subset` oraz wyniku zewnętrznego S_{ext} dla najlepszej konfiguracji każdej metody. Linia przerywana oznacza punkt kontrolny o najwyższym wyniku zewnętrznym.

Metody 02, 03 i 04 osiągnęły najlepszy wynik zewnętrzny w siódmym ocenianym punkcie kontrolnym, odpowiadającym krokowi 3000. Dalszy trening nie poprawiał już generalizacji, mimo że F1 na podzbiorze treningowym pozostawało bardzo wysokie. Metoda 01 dochodziła do najlepszego wyniku później, w końcowej części treningu. Pod względem liczby punktów kontrolnych potrzebnych do uzyskania najlepszego wyniku warianty generatywne i wariant następnego tokenu zbiegały więc szybciej niż klasyfikacja sekwencji.

Wśród kompletnie ocenionych wariantów najlepszym wyborem pozostaje metoda 03. Jej przewaga liczbowa nad metodami 02 i 04 jest mała, dlatego decyzja nie wynika wyłącznie z różnicy w S_{ext} . Istotne jest również to, że wariant następnego tokenu łączy wysoką skuteczność z prostą inferencją i nie wymaga parsera odpowiedzi. W dalszych eksperymentach stanowi więc najbardziej uzasadniony punkt odniesienia dla klasyfikacji `ham/spam`.

Bibliografia

- [1] An Yang i in. „Qwen3 Technical Report”. W: *arXiv preprint arXiv:2505.09388* (2025). URL: <https://arxiv.org/abs/2505.09388> (term. wiz. 07.06.2026).
- [2] Qwen Team. *Qwen3-0.6B Model Card*. URL: <https://huggingface.co/Qwen/Qwen3-0.6B> (term. wiz. 07.06.2026).
- [3] Google DeepMind. *Gemma 3 Model Card*. URL: https://ai.google.dev/gemma/docs/core/model_card_3 (term. wiz. 07.06.2026).
- [4] Meta. *Llama 3.2 Model Card*. URL: https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md (term. wiz. 07.06.2026).
- [5] Hugging Face TB. *SmolLM2-1.7B Model Card*. URL: <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B> (term. wiz. 07.06.2026).
- [6] Marah Abdin i in. „Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone”. W: *arXiv preprint arXiv:2404.14219* (2024). URL: <https://arxiv.org/abs/2404.14219> (term. wiz. 07.06.2026).
- [7] Microsoft. *Phi-3-mini-4k-instruct Model Card*. URL: <https://huggingface.co/microsoft/Phi-3-mini-4k-instruct> (term. wiz. 07.06.2026).
- [8] BenchGecko. *MMLU-PRO Benchmark: Llama 3.2 1B Instruct*. URL: <https://benchgecko.ai/benchmark/hf-mmlu-pro> (term. wiz. 07.06.2026).
- [9] *Enron Email Dataset*. URL: <https://www.kaggle.com/datasets/wcukierski/enron-email-dataset> (term. wiz. 27.05.2026).
- [10] *Emails for Spam or Ham Classification TREC 2007*. URL: <https://www.kaggle.com/datasets/bayes2003/emails-for-spam-or-ham-classification-trec-2007> (term. wiz. 27.05.2026).
- [11] *CEAS 08*. URL: <https://www.kaggle.com/datasets/doryanay/ceas-08> (term. wiz. 27.05.2026).
- [12] *Spam Assassin Email Classification Dataset*. URL: <https://www.kaggle.com/datasets/ganiyuolalekan/spam-assassin-email-classification-dataset> (term. wiz. 27.05.2026).
- [13] *Spam Mails Dataset*. URL: <https://www.kaggle.com/datasets/venky73/spam-mail-s-dataset> (term. wiz. 27.05.2026).

- [14] *Fraudulent Email Corpus*. URL: <https://www.kaggle.com/datasets/rtatman/fraudulent-email-corpus> (term. wiz. 27.05.2026).
- [15] Farooq A. Kperogi. „Your English Is Suspect”: Language, Communication, and the Pathologization of Nigerian Cyber Identity Through the Stylistic Imprints of Nigerian E-Mail Scams”. W: *Sage Journals* 42.1 (2018).
- [16] *Nazario 5 Datatest*. URL: <https://www.kaggle.com/datasets/zeyadkarimfcai/nazario-5-datatest> (term. wiz. 27.05.2026).
- [17] *Ling-Spam Dataset*. URL: <https://www.kaggle.com/datasets/mandygu/lingspam-dataset> (term. wiz. 27.05.2026).
- [18] Mengnan Du i in. „Shortcut Learning of Large Language Models in Natural Language Understanding”. W: *arxiv* 1.3 (2022).
- [19] *Advance Fee Fraud*. URL: <https://www.investor.gov/protect-your-investments/fraud/types-fraud/advance-fee-fraud>.